Assignment_2

March 4, 2026

1 Assignment 2 — Neural Sequence Models: From Recurrence to Self-Attention

---

1.1 Introduction

1.1.1 Recap: Assignment 1 Baseline Performance

Assignment 1 established classical machine learning baselines for **EXIST 2025 — Task 1, Subtask 1.1 (Sexism Identification in Tweets)**, a binary text classification task on a bilingual (English/Spanish) corpus of ~10,000 tweets. Using TF-IDF feature representations paired with linear classifiers (Logistic Regression and LinearSVC), and FastText mean-pooled embeddings as an alternative dense representation, the following results were obtained on the held-out validation set (1,038 samples):

Model	Feature Representation	CV F1-Macro	Val F1-Macro	Val Accuracy
Logistic Regression *(best overall)*	TF-IDF (unigrams, sublinear_tf)	0.6536	**0.75**	0.75
LinearSVC	TF-IDF (unigrams)	0.6524	—	—
SVM (LinearSVC) *(best dense)*	FastText embeddings (scratch)	0.5322	0.62	0.62
Logistic Regression	FastText embeddings (scratch)	0.5234	—	—

Key takeaways from Assignment 1:

- **Simple lowercasing** was the optimal preprocessing strategy (F1: 0.6506), outperforming stemming, lemmatization, and stopword removal.
- **Unigrams only** outperformed all higher-order n-gram configurations, due to feature space explosion relative to the small training corpus (~6,920 samples).

- **Sparse TF-IDF representations decisively outperformed dense FastText embeddings** trained from scratch (~12.1 F1-point gap), highlighting the limitations of mean-pooled embeddings on a small, domain-specific corpus.
- **Error analysis** revealed two systematic failure modes of bag-of-words models: (1) false negatives on tweets that *report or condemn* sexism (anti-sexist tone masks SEXIST label), and (2) false positives driven by gender-related keywords used in neutral or non-sexist contexts (e.g., references to *patriarchy* or *mgtow* without sexist intent). These errors share a common root cause: TF-IDF cannot model *how* words are used, only *which* words appear.

1.1.2 Research Questions

Assignment 1 demonstrated the ceiling of bag-of-words approaches for this task. This assignment transitions to **neural sequence models** — architectures that explicitly model sequential structure, word order, and long-range dependencies. Concretely, we implement and evaluate a **Bidirectional LSTM** and a **Transformer-based model** (fine-tuned from a pre-trained checkpoint, given the small dataset size), comparing them against the Assignment 1 baselines.

This motivates the following research questions:

> **Performance:** Do neural sequence models (BiLSTM, Transformer/BERT) yield meaningful F1-Macro improvements over the TF-IDF + Logistic Regression baseline on sexism identification in tweets?
>
> **Error correction:** Which specific error types from Assignment 1 — irony/sarcasm, anti-sexist reporting, keyword mismatch — are resolved by architectures that capture sequential context and pragmatic meaning?
>
> **Data efficiency:** Given the limited training size (~6,920 samples), do neural models trained from scratch suffer from data starvation, and does transfer learning via a pre-trained Transformer (e.g., multilingual BERT / XLM-R) mitigate this?
>
> **Computational cost:** Is the added complexity of neural models — in training time, GPU memory, and inference latency — justified by the performance gains over the classical baseline?
>
> **Architecture trade-offs:** How do BiLSTM and Transformer architectures compare in terms of their inductive biases for this specific task (short bilingual tweets, subtle/ironic content, near-balanced classes)?

[1]:
```
import json
import pandas as pd
import numpy as np
import re
import nltk
import matplotlib.pyplot as plt
import seaborn as sns
import ast
from time import time, sleep, perf_counter
import torch
```

```python
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, Subset

# Sklearn imports
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import classification_report, confusion_matrix, f1_score

# NLTK downloads
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)
nltk.download('punkt', quiet=True)

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Configure seed for reproducibility
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

1.2 Data Ingestion and Preprocessing

In this section, we handle the ingestion of the JSON datasets containing tweets in both English and Spanish. To establish a robust ground truth, we apply a majority voting strategy that resolves discrepancies among different annotators, providing a definitive label for each text. Unlike traditional bag-of-words models evaluated in the first assignment, where extensive text cleaning is beneficial, neural sequence models rely heavily on the original semantic context and syntactic structure. For this reason, we keep preprocessing to an absolute minimum, applying only lowercasing. This approach allows us to standardize the vocabulary without sacrificing punctuation marks or stopwords, which provide vital sequential information for the model's contextual learning.

[2]:
```python
def load_and_parse_data(filepath):
    """
    Parses nested JSON and applies Majority Voting for labels.
    """
    with open(filepath, 'r', encoding='utf-8') as f:
        data = json.load(f)

    df = pd.DataFrame.from_dict(data, orient='index')
    df = df.reset_index(drop=True).rename(columns={'index': 'id_EXIST'})

    # Label Processing (Majority Voting)
    if 'labels_task1_1' in df.columns:
        def get_majority_vote(labels_list):
```

```
            if not isinstance(labels_list, list): return np.nan
            counts = pd.Series(labels_list).value_counts()
            # Tie-breaking: Prioritize 'YES' (Sexism) if tie
            if len(counts) > 1 and counts.iloc[0] == counts.iloc[1]:
                if 'YES' in counts.index[:2]: return 'YES'
            return counts.idxmax()

        df['final_label_str'] = df['labels_task1_1'].apply(get_majority_vote)
        df['label'] = df['final_label_str'].map({'YES': 1, 'NO': 0})
        df = df.dropna(subset=['label'])
        df['label'] = df['label'].astype(int)

    return df

print("Loading Data...")
df_train = load_and_parse_data('../data/training/EXIST2025_training.json')
df_val = load_and_parse_data('../data/dev/EXIST2025_dev.json')
df_test = load_and_parse_data('../data/test/EXIST2025_test_clean.json')

print(f"\nTotal Samples - Training: {len(df_train)}")
print(df_train['final_label_str'].value_counts())
print(f"\nTotal Samples - Validation (Will be used as TEST): {len(df_val)}")
print(df_val['final_label_str'].value_counts())
```

```
Loading Data…

Total Samples - Training: 6920
final_label_str
YES    3553
NO     3367
Name: count, dtype: int64

Total Samples - Validation (Will be used as TEST): 1038
final_label_str
YES    559
NO     479
Name: count, dtype: int64
```

[3]:
```
stop_words = set(stopwords.words('english')) | set(stopwords.words('spanish'))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def preprocess_text(text, strategy='raw'):
    text_processed = str(text)
    if strategy == 'raw':
        return text_processed
    if strategy == 'lowercase':
```

```
        return text_processed.lower()
    if strategy == 'no_punct':
        text_processed = re.sub(r'[^\w\s]', '', text_processed)
        return text_processed.lower()
    if strategy == 'no_stopwords':
        text_processed = text_processed.lower()
        words = text_processed.split()
        return " ".join([w for w in words if w not in stop_words])
    if strategy == 'stemmed':
        text_processed = text_processed.lower()
        words = text_processed.split()
        return " ".join([stemmer.stem(w) for w in words])
    if strategy == 'lemmatized':
        text_processed = text_processed.lower()
        words = text_processed.split()
        return " ".join([lemmatizer.lemmatize(w) for w in words])
    return text_processed

# Process texts
df_train['text_clean'] = df_train['tweet'].apply(lambda x: preprocess_text(x,␣
 ↪'lowercase'))
df_val['text_clean'] = df_val['tweet'].apply(lambda x: preprocess_text(x,␣
 ↪'lowercase'))
df_test['text_clean'] = df_test['tweet'].apply(lambda x: preprocess_text(x,␣
 ↪'lowercase'))
```

1.3 Bidirectional LSTM (BiLSTM) Architecture

To capture the purely sequential nature of text, we implemented a Bidirectional Long Short-Term Memory (BiLSTM) network. The first step in this modeling process is building a custom vocabulary from our training set, assigning unique numerical indices to each word. Unknown terms appearing in validation or testing are grouped under a special token, and sequences are padded to maintain a uniform input length across tensors. Regarding the model design, the BiLSTM architecture processes the dense word embeddings in both directions, from left to right and vice versa. This bidirectional capability allows the network to understand the complete context of a word based on its entire lexical environment, rather than just the preceding words. Subsequently, the hidden output from this recurrent layer is passed to a fully connected linear layer that acts as the final classifier, determining whether the tweet contains sexist language. Furthermore, to mitigate the high risk of overfitting that these complex networks suffer when trained on small datasets, we integrated an Early Stopping mechanism that halts the training process the exact moment the validation F1-score stops improving.

[4]:
```
class Vocabulary:
    def __init__(self, min_freq=2):
        self.itos = {0: "<PAD>", 1: "<UNK>"}
        self.stoi = {"<PAD>": 0, "<UNK>": 1}
        self.min_freq = min_freq
```

```
    def build_vocabulary(self, sentence_list):
        frequencies = {}
        idx = 2
        for sentence in sentence_list:
            for word in self.tokenize(sentence):
                if word not in frequencies:
                    frequencies[word] = 1
                else:
                    frequencies[word] += 1
                if frequencies[word] == self.min_freq:
                    self.stoi[word] = idx
                    self.itos[idx] = word
                    idx += 1

    def tokenize(self, text):
        return re.findall(r'\w+', text)

    def numericalize(self, text):
        tokenized_text = self.tokenize(text)
        return [self.stoi.get(token, self.stoi["<UNK>"]) for token in␣
 ↪tokenized_text]

vocab = Vocabulary(min_freq=2)
vocab.build_vocabulary(df_train['text_clean'].tolist())
```

[5]:
```
class EXISTDataset(Dataset):
    def __init__(self, df, vocab, max_len=64):
        self.df = df
        self.vocab = vocab
        self.max_len = max_len

    def __len__(self):
        return len(self.df)

    def __getitem__(self, index):
        text = self.df.iloc[index]['text_clean']
        label = self.df.iloc[index]['label'] if 'label' in self.df.columns else␣
 ↪-1

        tokens = self.vocab.numericalize(text)
        length = len(tokens)

        # Evitar secuencias vacías (rompen el pack_padded_sequence)
        if length == 0:
            tokens = [self.vocab.stoi["<UNK>"]]
            length = 1
```

```
        # Truncar si es más largo que max_len
        if length > self.max_len:
            tokens = tokens[:self.max_len]
            length = self.max_len

        # Rellenar (Padding)
        padded_tokens = tokens + [self.vocab.stoi["<PAD>"]] * (self.max_len -␣
↪length)

        return torch.tensor(padded_tokens), torch.tensor(label, dtype=torch.
↪long), torch.tensor(length, dtype=torch.long)

# Dataset global de Train (se dividirá en Folds)
train_dataset = EXISTDataset(df_train, vocab)

# El dataset "dev" original será nuestro "TEST" real
test_dataset_real = EXISTDataset(df_val, vocab)
test_loader_real = DataLoader(test_dataset_real, batch_size=32, shuffle=False)
```

1.3.1 Hyperparameter Optimization with Optuna and MLflow

Training deep recurrent neural networks from scratch on relatively small datasets is highly prone to overfitting. To solve this problem and find the optimal architectural configuration for our BiLSTM, we employ the Optuna library, automating the hyperparameter search across multiple trials. To ensure robust evaluation, we use a 5-fold Stratified Cross-Validation strategy. Simultaneously, this entire process, along with the resulting metrics and configurations, is tracked and centrally logged using MLflow. During this optimization process, the search space dynamically explores several critical dimensions of the model. Structural parameters such as the embedding dimension, the network's hidden dimension, and its depth are adjusted by varying the number of stacked layers. Similarly, the algorithm searches for the ideal dropout rate, which acts as a fundamental regularization mechanism by randomly dropping neurons, and the learning rate, dictating the magnitude of weight updates during gradient descent.

[6]:
```
import optuna
import mlflow
import mlflow.pytorch
from optuna.integration.mlflow import MLflowCallback

class BiLSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,␣
↪n_layers, dropout=0.5):
        super(BiLSTMClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=n_layers,
                            bidirectional=True, batch_first=True,
                            dropout=dropout if n_layers > 1 else 0)
```

```python
        self.fc = nn.Linear(hidden_dim * 2, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, text, lengths):
        embedded = self.dropout(self.embedding(text))
        packed = nn.utils.rnn.pack_padded_sequence(
            embedded, lengths.cpu(), batch_first=True, enforce_sorted=False
        )
        packed_output, (hidden, cell) = self.lstm(packed)
        forward_hidden = hidden[-2]
        backward_hidden = hidden[-1]
        final_hidden = torch.cat([forward_hidden, backward_hidden], dim=1)
        output = self.dropout(final_hidden)
        logits = self.fc(output)
        return logits


class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_score = None
        self.early_stop = False
        self.best_model_state = None

    def __call__(self, val_score, model):
        if self.best_score is None:
            self.best_score = val_score
            self.best_model_state = model.state_dict().copy()
        elif val_score < self.best_score + self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_score = val_score
            self.best_model_state = model.state_dict().copy()
            self.counter = 0
        return self.early_stop


# Fixed constants
VOCAB_SIZE = len(vocab.stoi)
OUTPUT_DIM = 2
EPOCHS = 20
N_SPLITS = 5
SEED = 42
```

```
BATCH_SIZE = 512
```

[7]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")

# MLflow setup
MLFLOW_EXPERIMENT = "BiLSTM_Optuna_Tuning"
mlflow.set_experiment(MLFLOW_EXPERIMENT)

# Search space
# All hyperparameters Optuna will tune are defined here for easy editing.
SEARCH_SPACE = {
    "embedding_dim" : ("categorical", [100, 200, 300]),
    "hidden_dim" : ("categorical", [128, 256, 512]),
    "n_layers" : ("int",          [1, 3]), # suggest_int(low, high)
    "dropout" : ("float",        [0.2, 0.5]), # suggest_float(low, high)
    "lr" : ("loguniform",  [1e-4, 1e-2]) # suggest_float(..., log=True)
}


def suggest_hyperparams(trial):
    """Translate SEARCH_SPACE into Optuna suggestions."""
    params = {}
    for name, (kind, bounds) in SEARCH_SPACE.items():
        if kind == "categorical":
            params[name] = trial.suggest_categorical(name, bounds)
        elif kind == "int":
            params[name] = trial.suggest_int(name, *bounds)
        elif kind == "float":
            params[name] = trial.suggest_float(name, *bounds)
        elif kind == "loguniform":
            params[name] = trial.suggest_float(name, *bounds, log=True)
    return params


def run_cv(params: dict) -> tuple[float, dict]:
    """
    5-fold stratified CV for a given hyperparameter configuration.
    Returns (mean_val_f1, per-fold metrics dict).
    """
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
    y_array = df_train['label'].values
    fold_f1s = []

    for fold, (train_idx, val_idx) in enumerate(
        skf.split(np.zeros(len(y_array)), y_array)
    ):
```

```python
train_loader = DataLoader(
    Subset(train_dataset, train_idx),
    batch_size=BATCH_SIZE, shuffle=True
)
val_loader = DataLoader(
    Subset(train_dataset, val_idx),
    batch_size=BATCH_SIZE, shuffle=False
)

model = BiLSTMClassifier(
    vocab_size = VOCAB_SIZE,
    embedding_dim = params["embedding_dim"],
    hidden_dim = params["hidden_dim"],
    output_dim = OUTPUT_DIM,
    n_layers = params["n_layers"],
    dropout = params["dropout"],
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=params["lr"])
early_stopping = EarlyStopping(patience=5)

for epoch in range(EPOCHS):
    # Train
    model.train()
    for texts, labels, lengths in train_loader:
        texts, labels = texts.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(texts, lengths), labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

    # Validate
    model.eval()
    preds, targets = [], []
    with torch.no_grad():
        for texts, labels, lengths in val_loader:
            texts = texts.to(device)
            out = model(texts, lengths)
            preds.extend(out.argmax(1).cpu().numpy())
            targets.extend(labels.numpy())

    val_f1 = f1_score(targets, preds, average='macro')

    if early_stopping(val_f1, model):
        break
```

```
        fold_f1s.append(early_stopping.best_score)

    mean_f1 = float(np.mean(fold_f1s))
    per_fold_log = {f"fold_{i+1}_f1": v for i, v in enumerate(fold_f1s)}
    return mean_f1, per_fold_log


def objective(trial: optuna.Trial) -> float:
    """Optuna objective: suggest params → CV → log to MLflow → return metric."""
    params = suggest_hyperparams(trial)

    with mlflow.start_run(
        run_name=f"trial_{trial.number}", nested=True
    ):
        # Log all hyperparameters
        mlflow.log_params(params)
        mlflow.log_param("trial_number", trial.number)

        mean_f1, per_fold_log = run_cv(params)

        # Log per-fold and aggregate metrics
        mlflow.log_metrics(per_fold_log)
        mlflow.log_metric("mean_cv_f1_macro", mean_f1)

        # Optuna pruning hook (optional but useful with a Pruner)
        trial.report(mean_f1, step=0)
        if trial.should_prune():
            raise optuna.exceptions.TrialPruned()

    return mean_f1
```

```
Device: cuda
```

```
2026/03/04 16:24:49 INFO mlflow.store.db.utils: Creating initial MLflow database
tables…
2026/03/04 16:24:49 INFO mlflow.store.db.utils: Updating database tables
2026/03/04 16:24:50 INFO mlflow.tracking.fluent: Experiment with name
'BiLSTM_Optuna_Tuning' does not exist. Creating a new experiment.
```

[8]:
```
STUDY_NAME = "bilstm_sexism"
STORAGE_URL = "sqlite:///optuna_bilstm.db"   # persisted to disk
```

[9]:
```
torch.manual_seed(SEED)
np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

```python
with mlflow.start_run(run_name="optuna_study"):
    sampler = optuna.samplers.TPESampler(seed=SEED)
    pruner = optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=0)
    study = optuna.create_study(
        direction = "maximize",
        sampler = sampler,
        pruner = pruner,
        study_name = STUDY_NAME,
        storage = STORAGE_URL,
        load_if_exists = True,
    )
    study.optimize(
        objective,
        n_trials = 15,
        timeout = None,
        show_progress_bar = True,
    )
    best = study.best_trial
    mlflow.log_params({f"best_{k}": v for k, v in best.params.items()})
    mlflow.log_metric("best_mean_cv_f1_macro", best.value)
    print("\n" + "=" * 50)
    print(f"Best trial:  #{best.number}")
    print(f"Best CV F1-Macro: {best.value:.4f}")
    print("Best hyperparameters:")
    for k, v in best.params.items():
        print(f"  {k}: {v}")
```

```
[I 2026-03-04 16:24:50,828] A new study created in RDB with name:
bilstm_sexism

  0%|          | 0/15 [00:00<?, ?it/s]

[I 2026-03-04 16:26:30,155] Trial 0 finished with value:
0.7137501946393671 and parameters: {'embedding_dim': 200, 'hidden_dim': 128,
'n_layers': 1, 'dropout': 0.45985284373248053, 'lr': 0.0015930522616241021}.
Best is trial 0 with value: 0.7137501946393671.
[I 2026-03-04 16:27:47,890] Trial 1 finished with value:
0.7060093519534321 and parameters: {'embedding_dim': 300, 'hidden_dim': 128,
'n_layers': 1, 'dropout': 0.2912726728878613, 'lr': 0.0011207606211860567}. Best
is trial 0 with value: 0.7137501946393671.
[I 2026-03-04 16:29:42,683] Trial 2 finished with value:
0.7015422365423276 and parameters: {'embedding_dim': 300, 'hidden_dim': 512,
'n_layers': 2, 'dropout': 0.43555278841790407, 'lr': 0.00025081156860452336}.
Best is trial 0 with value: 0.7137501946393671.
[I 2026-03-04 16:31:05,618] Trial 3 finished with value:
0.7178342020643153 and parameters: {'embedding_dim': 200, 'hidden_dim': 128,
'n_layers': 3, 'dropout': 0.4896896099223678, 'lr': 0.004138040112561018}. Best
is trial 3 with value: 0.7178342020643153.
[I 2026-03-04 16:32:49,015] Trial 4 finished with value:
```

```
0.704457631050545 and parameters: {'embedding_dim': 300, 'hidden_dim': 512,
'n_layers': 1, 'dropout': 0.47279612062363463, 'lr': 0.00032927591344236165}.
Best is trial 3 with value: 0.7178342020643153.
[I 2026-03-04 16:35:21,715] Trial 5 pruned.
[I 2026-03-04 16:37:08,023] Trial 6 pruned.
[I 2026-03-04 16:39:06,714] Trial 7 finished with value:
0.7102541253933149 and parameters: {'embedding_dim': 300, 'hidden_dim': 256,
'n_layers': 3, 'dropout': 0.4316734307889972, 'lr': 0.00024970737145052745}.
Best is trial 3 with value: 0.7178342020643153.
[I 2026-03-04 16:40:12,110] Trial 8 pruned.
[I 2026-03-04 16:42:11,054] Trial 9 pruned.
[I 2026-03-04 16:43:50,052] Trial 10 finished with value:
0.7131550548860568 and parameters: {'embedding_dim': 200, 'hidden_dim': 128,
'n_layers': 3, 'dropout': 0.3801285454653292, 'lr': 0.0023591442255668847}. Best
is trial 3 with value: 0.7178342020643153.
[I 2026-03-04 16:45:13,796] Trial 11 finished with value:
0.7118749788698951 and parameters: {'embedding_dim': 200, 'hidden_dim': 128,
'n_layers': 1, 'dropout': 0.37917536837740246, 'lr': 0.0018652214300202052}.
Best is trial 3 with value: 0.7178342020643153.
[I 2026-03-04 16:46:28,547] Trial 12 finished with value:
0.7164831565069655 and parameters: {'embedding_dim': 200, 'hidden_dim': 128,
'n_layers': 3, 'dropout': 0.4974950548087196, 'lr': 0.009129978603227817}. Best
is trial 3 with value: 0.7178342020643153.
[I 2026-03-04 16:47:37,562] Trial 13 pruned.
[I 2026-03-04 16:49:07,755] Trial 14 pruned.

==================================================
Best trial:  #3
Best CV F1-Macro: 0.7178
Best hyperparameters:
  embedding_dim: 200
  hidden_dim: 128
  n_layers: 3
  dropout: 0.4896896099223678
  lr: 0.004138040112561018
```

[10]:
```python
torch.manual_seed(SEED)
np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# Load study from disk
study = optuna.load_study(study_name=STUDY_NAME, storage=STORAGE_URL)
best_params = study.best_trial.params

print("Loaded best hyperparameters:")
for k, v in best_params.items():
```

```
    print(f"  {k}: {v}")

with mlflow.start_run(run_name="final_model_best_params"):
    mlflow.log_params(best_params)

    final_model = BiLSTMClassifier(
        vocab_size = VOCAB_SIZE,
        embedding_dim = best_params["embedding_dim"],
        hidden_dim = best_params["hidden_dim"],
        output_dim = OUTPUT_DIM,
        n_layers = best_params["n_layers"],
        dropout = best_params["dropout"],
    ).to(device)

    # Print model parameters
    print("\nModel architecture:")
    print(final_model)
    total_params = sum(p.numel() for p in final_model.parameters())
    trainable_params = sum(p.numel() for p in final_model.parameters() if p.
 ↪requires_grad)
    print(f"\nTotal parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")
    mlflow.log_param("total_params", total_params)
    mlflow.log_param("trainable_params", trainable_params)

    full_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(final_model.parameters(), lr=best_params["lr"])

    epoch_times = []
    total_train_start = perf_counter()

    for epoch in range(EPOCHS):
        epoch_start = perf_counter()

        final_model.train()
        train_loss = 0
        all_preds, all_labels = [], []

        for texts, labels, lengths in full_loader:
            texts, labels = texts.to(device), labels.to(device)
            optimizer.zero_grad()
            logits = final_model(texts, lengths)
            loss = criterion(logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(final_model.parameters(), max_norm=1.
 ↪0)
```

```
            optimizer.step()
            train_loss += loss.item()
            all_preds.extend(logits.detach().argmax(1).cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

        epoch_time = perf_counter() - epoch_start
        epoch_times.append(epoch_time)

        train_f1 = f1_score(all_labels, all_preds, average='macro')
        mlflow.log_metric("train_f1_macro", train_f1, step=epoch)
        mlflow.log_metric("train_loss", train_loss / len(full_loader),␣
↪step=epoch)
        mlflow.log_metric("epoch_time_sec", epoch_time, step=epoch)

        print(f"Epoch {epoch+1:02d}/{EPOCHS} | "
              f"Loss: {train_loss/len(full_loader):.4f} | "
              f"F1: {train_f1:.4f} | "
              f"Time: {epoch_time:.2f}s")

    total_time = perf_counter() - total_train_start
    mean_epoch_time = sum(epoch_times) / len(epoch_times)

    mlflow.log_metric("total_train_time_sec", total_time)
    mlflow.log_metric("mean_epoch_time_sec", mean_epoch_time)

    print(f"\nTotal training time: {total_time:.2f}s")
    print(f"Mean training time/epoch: {mean_epoch_time:.2f}s")

    mlflow.pytorch.log_model(final_model, artifact_path="bilstm_final_model")
    print("\nFinal model trained and logged to MLflow.")
```

```
Loaded best hyperparameters:
  embedding_dim: 200
  hidden_dim: 128
  n_layers: 3
  dropout: 0.4896896099223678
  lr: 0.004138040112561018

Model architecture:
BiLSTMClassifier(
  (embedding): Embedding(11013, 200, padding_idx=0)
  (lstm): LSTM(200, 128, num_layers=3, batch_first=True,
dropout=0.4896896099223678, bidirectional=True)
  (fc): Linear(in_features=256, out_features=2, bias=True)
  (dropout): Dropout(p=0.4896896099223678, inplace=False)
)

Total parameters: 3,331,562
```

```
Trainable parameters: 3,331,562
Epoch 01/20 | Loss: 0.6817 | F1: 0.5542 | Time: 1.41s
Epoch 02/20 | Loss: 0.6294 | F1: 0.6404 | Time: 1.41s
Epoch 03/20 | Loss: 0.5894 | F1: 0.6936 | Time: 1.44s
Epoch 04/20 | Loss: 0.5449 | F1: 0.7251 | Time: 1.41s
Epoch 05/20 | Loss: 0.4922 | F1: 0.7631 | Time: 1.41s
Epoch 06/20 | Loss: 0.4494 | F1: 0.7936 | Time: 1.44s
Epoch 07/20 | Loss: 0.3862 | F1: 0.8288 | Time: 1.41s
Epoch 08/20 | Loss: 0.3525 | F1: 0.8486 | Time: 1.41s
Epoch 09/20 | Loss: 0.3059 | F1: 0.8680 | Time: 1.41s
Epoch 10/20 | Loss: 0.2771 | F1: 0.8842 | Time: 1.40s
Epoch 11/20 | Loss: 0.2505 | F1: 0.8927 | Time: 1.41s
Epoch 12/20 | Loss: 0.2155 | F1: 0.9128 | Time: 1.41s
Epoch 13/20 | Loss: 0.2038 | F1: 0.9210 | Time: 1.43s
Epoch 14/20 | Loss: 0.1736 | F1: 0.9328 | Time: 1.45s
Epoch 15/20 | Loss: 0.1519 | F1: 0.9414 | Time: 1.41s
Epoch 16/20 | Loss: 0.1286 | F1: 0.9507 | Time: 1.40s
Epoch 17/20 | Loss: 0.1319 | F1: 0.9487 | Time: 1.43s
Epoch 18/20 | Loss: 0.1350 | F1: 0.9482 | Time: 1.43s
Epoch 19/20 | Loss: 0.1198 | F1: 0.9530 | Time: 1.42s

2026/03/04 16:49:37 WARNING mlflow.models.model: `artifact_path` is deprecated.
Please use `name` instead.
2026/03/04 16:49:37 WARNING mlflow.pytorch: Saving pytorch model by Pickle or
CloudPickle format requires exercising caution because these formats rely on
Python's object serialization mechanism, which can execute arbitrary code during
deserialization.The recommended safe alternative is to set 'export_model' to
True to save the pytorch model using the safe graph model format.

Epoch 20/20 | Loss: 0.1131 | F1: 0.9560 | Time: 1.41s


Total training time: 29.12s
Mean training time/epoch: 1.42s

2026/03/04 16:49:43 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.


Final model trained and logged to MLflow.
```

[11]:
```python
print("Evaluating the final model on the TEST set (original 'dev' set)...")

with mlflow.start_run(run_name="final_model_test_evaluation"):
    mlflow.log_params(best_params)

    final_model.eval()
    test_preds, test_labels_list = [], []
```

```
    start_time = time()
    with torch.no_grad():
        for texts, labels, lengths in test_loader_real:
            texts, labels = texts.to(device), labels.to(device)
            outputs = final_model(texts, lengths)
            test_preds.extend(outputs.argmax(1).cpu().numpy())
            test_labels_list.extend(labels.cpu().numpy())
    inference_time = time() - start_time

    test_f1 = f1_score(test_labels_list, test_preds, average='macro')

    # Log metrics to MLflow
    mlflow.log_metric("test_f1_macro", test_f1)
    mlflow.log_metric("inference_time_seconds", inference_time)

    # Console output
    print(f"\nFinal F1-Macro (Test Set): {test_f1:.4f}")
    print(f"Total Inference Time: {inference_time:.3f} seconds")
    print("\nConfusion Matrix:\n",
          confusion_matrix(test_labels_list, test_preds))
    print("\nClassification Report:\n",
          classification_report(test_labels_list, test_preds))
```

```
Evaluating the final model on the TEST set (original 'dev' set)…

Final F1-Macro (Test Set): 0.7177
Total Inference Time: 0.556 seconds

Confusion Matrix:
 [[343 136]
 [156 403]]

Classification Report:
               precision    recall  f1-score   support

           0       0.69      0.72      0.70       479
           1       0.75      0.72      0.73       559

    accuracy                           0.72      1038
   macro avg       0.72      0.72      0.72      1038
weighted avg       0.72      0.72      0.72      1038
```

[12]:
```
# Store results
bilstm_test_preds = list(test_preds)
bilstm_test_labels = list(test_labels_list)
bilstm_test_f1 = test_f1
```

```
bilstm_inf_time = inference_time
bilstm_total_time = total_time
bilstm_mean_epoch_time = mean_epoch_time
bilstm_total_params = total_params
bilstm_trainable_params = trainable_params
```

1.4 Transformer Models: BERT and DistilBERT

To definitively overcome the data scarcity issue suffered by recurrent models trained from scratch, we turn to Transfer Learning using pre-trained Transformer architectures. Given the bilingual nature of the EXIST 2025 corpus, we decided to evaluate two highly popular multilingual checkpoints hosted on HuggingFace. On one hand, we analyze the base version of BERT, a robust and deep model; on the other hand, we test DistilBERT, a distilled counterpart that offers a lighter architecture, retaining much of the original performance while reducing the computational load. Unlike previous approaches that rely on comprehensive dictionaries, Transformers utilize sub-word tokenization via WordPiece. This approach is highly effective, as it breaks down complex or unknown words into smaller fragments, naturally handling out-of-vocabulary issues. In our implementation, instead of freezing the model's base layers, we perform a full fine-tuning of the entire network. This means we modify all the pre-trained weights of the Transformer to adapt its rich, general semantic representations to the subtlety of our specific sexism detection task. To achieve stable convergence during this delicate process, we use the AdamW optimizer combined with a linear learning rate scheduler that includes a warmup phase.

[13]:
```
from transformers import AutoModel, AutoTokenizer,␣
↪get_linear_schedule_with_warmup
import torch.nn as nn
import torch
```

[14]:
```
class BERTDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts.reset_index(drop=True)
        self.labels = labels.reset_index(drop=True)
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, index):
        text = str(self.texts[index])
        label = self.labels[index]

        encoding = self.tokenizer(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            return_token_type_ids=False,
```

```
            padding='max_length',
            truncation=True,
            return_attention_mask=True,
            return_tensors='pt',
        )

        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'label': torch.tensor(label, dtype=torch.long)
        }
```

[15]:
```
class BERTClassifier(nn.Module):
    def __init__(self, model_name, num_classes, freeze_bert=False):
        super().__init__()
        self.bert = AutoModel.from_pretrained(model_name)
        self.dropout = nn.Dropout(0.3)
        self.classifier = nn.Linear(self.bert.config.hidden_size, num_classes)

        if freeze_bert:
            for param in self.bert.parameters():
                param.requires_grad = False

    def forward(self, input_ids, attention_mask):
        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        pooled = outputs.last_hidden_state[:, 0]
        pooled = self.dropout(pooled)
        logits = self.classifier(pooled)
        return logits

def unfreeze_last_n_layers(model, n):
    """Utility for Ablation Study"""
    # First, freeze all
    for param in model.bert.parameters():
        param.requires_grad = False

    # Unfreeze last n encoder layers
    if n > 0:
        for layer in model.bert.encoder.layer[-n:]:
            for param in layer.parameters():
                param.requires_grad = True

    # Always unfreeze classification head
    for param in model.classifier.parameters():
        param.requires_grad = True
```

1.4.1 BERT

[16]:
```
MODEL_NAME = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

[17]:
```
# Preparing raw texts
X_train_bert = df_train['tweet']
y_train_bert = df_train['label']

X_test_bert = df_val['tweet']
y_test_bert = df_val['label']
```

[18]:
```
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from torch.utils.data import Subset

torch.manual_seed(SEED)
np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Training BERT in: {device}\n")

EPOCHS_BERT = 4
BATCH_SIZE = 16
N_SPLITS = 5

skf_bert = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
y_train_array_bert = y_train_bert.values
bert_fold_metrics = []
best_bert_global_f1 = 0
best_bert_model_state = None

bert_full_dataset = BERTDataset(X_train_bert, y_train_bert, tokenizer)

for fold, (train_idx, val_idx) in enumerate(skf_bert.split(np.
 ↪zeros(len(y_train_array_bert)), y_train_array_bert)):
    print(f"================ BERT FOLD {fold + 1}/{N_SPLITS} ================")

    train_sub = Subset(bert_full_dataset, train_idx)
    val_sub = Subset(bert_full_dataset, val_idx)

    train_loader = DataLoader(train_sub, batch_size=BATCH_SIZE, shuffle=True)
    val_loader = DataLoader(val_sub, batch_size=BATCH_SIZE, shuffle=False)

    model = BERTClassifier(MODEL_NAME, num_classes=2, freeze_bert=False).
 ↪to(device)
```

```
    # Print parameter count once (first fold only)
    if fold == 0:
        bert_total_params = sum(p.numel() for p in model.parameters())
        bert_trainable_params = sum(p.numel() for p in model.parameters() if p.
↪requires_grad)
        print(f"\nModel architecture:")
        print(model)
        print(f"\nTotal parameters: {total_params:,}")
        print(f"Trainable parameters: {trainable_params:,}\n")

    optimizer = torch.optim.AdamW([
        {'params': model.bert.parameters(), 'lr': 2e-5},
        {'params': model.classifier.parameters(), 'lr': 1e-4}
    ])

    total_steps = len(train_loader) * EPOCHS_BERT
    warmup_steps = int(total_steps * 0.1)
    scheduler = get_linear_schedule_with_warmup(optimizer,␣
↪num_warmup_steps=warmup_steps, num_training_steps=total_steps)

    criterion = nn.CrossEntropyLoss()
    best_fold_f1 = 0
    epoch_times = []
    fold_start = perf_counter()

    for epoch in range(EPOCHS_BERT):
        epoch_start = perf_counter()

        model.train()
        train_loss = 0
        all_train_preds, all_train_labels = [], []

        for batch in train_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['label'].to(device)

            optimizer.zero_grad()
            logits = model(input_ids, attention_mask)
            loss = criterion(logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            scheduler.step()

            train_loss += loss.item()
```

```
            _, predicted = torch.max(logits, 1)
            all_train_preds.extend(predicted.cpu().numpy())
            all_train_labels.extend(labels.cpu().numpy())

        model.eval()
        val_loss = 0
        all_val_preds, all_val_labels = [], []

        with torch.no_grad():
            for batch in val_loader:
                input_ids = batch['input_ids'].to(device)
                attention_mask = batch['attention_mask'].to(device)
                labels = batch['label'].to(device)

                logits = model(input_ids, attention_mask)
                loss = criterion(logits, labels)

                val_loss += loss.item()
                _, predicted = torch.max(logits, 1)
                all_val_preds.extend(predicted.cpu().numpy())
                all_val_labels.extend(labels.cpu().numpy())

        epoch_time = perf_counter() - epoch_start
        epoch_times.append(epoch_time)

        val_f1 = f1_score(all_val_labels, all_val_preds, average='macro')
        print(f"Epoch {epoch+1}/{EPOCHS_BERT} | "
              f"Train Loss: {train_loss/len(train_loader):.4f} | "
              f"Val Loss: {val_loss/len(val_loader):.4f} | "
              f"Val F1: {val_f1:.4f} | "
              f"Time: {epoch_time:.2f}s")

        if val_f1 > best_fold_f1:
            best_fold_f1 = val_f1
            best_model_state_for_this_fold = model.state_dict().copy()

    fold_time = perf_counter() - fold_start
    bert_mean_epoch_time = sum(epoch_times) / len(epoch_times)

    bert_fold_metrics.append(best_fold_f1)
    print(f"\nBest F1-Macro in Fold {fold+1}: {best_fold_f1:.4f}")
    print(f"Fold total time: {fold_time:.2f}s | Mean time/epoch:␣
↪{bert_mean_epoch_time:.2f}s\n")

    if best_fold_f1 > best_bert_global_f1:
        best_bert_global_f1 = best_fold_f1
        best_bert_model_state = best_model_state_for_this_fold
```

```python
print("=" * 40)
print(f"Mean F1-Macro for BERT (5 Folds): {np.mean(bert_fold_metrics):.4f}")
print("=" * 40)
```

```
Training BERT in: cuda

================ BERT FOLD 1/5 ================

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.


Model architecture:
BERTClassifier(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(119547, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
```

```
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.3, inplace=False)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
)

Total parameters: 3,331,562
Trainable parameters: 3,331,562

Epoch 1/4 | Train Loss: 0.6040 | Val Loss: 0.4627 | Val F1: 0.7858 | Time:
83.35s
Epoch 2/4 | Train Loss: 0.4211 | Val Loss: 0.4441 | Val F1: 0.7964 | Time:
83.39s
Epoch 3/4 | Train Loss: 0.2651 | Val Loss: 0.5415 | Val F1: 0.7896 | Time:
83.26s
Epoch 4/4 | Train Loss: 0.1404 | Val Loss: 0.8199 | Val F1: 0.7902 | Time:
83.38s

Best F1-Macro in Fold 1: 0.7964
Fold total time: 333.40s | Mean time/epoch: 83.35s

================ BERT FOLD 2/5 ================

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
```

```
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.6002 | Val Loss: 0.5281 | Val F1: 0.7226 | Time:
83.40s
Epoch 2/4 | Train Loss: 0.4221 | Val Loss: 0.5009 | Val F1: 0.7752 | Time:
75.03s
Epoch 3/4 | Train Loss: 0.2743 | Val Loss: 0.5962 | Val F1: 0.7768 | Time:
73.59s
Epoch 4/4 | Train Loss: 0.1496 | Val Loss: 0.9409 | Val F1: 0.7753 | Time:
73.49s

Best F1-Macro in Fold 2: 0.7768
Fold total time: 305.52s | Mean time/epoch: 76.38s

================ BERT FOLD 3/5 ================

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.6067 | Val Loss: 0.4963 | Val F1: 0.7695 | Time:
73.55s
Epoch 2/4 | Train Loss: 0.4376 | Val Loss: 0.5134 | Val F1: 0.7781 | Time:
73.58s
Epoch 3/4 | Train Loss: 0.2971 | Val Loss: 0.5543 | Val F1: 0.7883 | Time:
73.64s
Epoch 4/4 | Train Loss: 0.1932 | Val Loss: 0.7559 | Val F1: 0.7811 | Time:
73.60s
```

```
Best F1-Macro in Fold 3: 0.7883
Fold total time: 294.40s | Mean time/epoch: 73.59s

================ BERT FOLD 4/5 ================

Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.5933 | Val Loss: 0.4697 | Val F1: 0.7743 | Time:
73.61s
Epoch 2/4 | Train Loss: 0.4163 | Val Loss: 0.4780 | Val F1: 0.7796 | Time:
73.52s
Epoch 3/4 | Train Loss: 0.2610 | Val Loss: 0.5797 | Val F1: 0.7862 | Time:
73.63s
Epoch 4/4 | Train Loss: 0.1429 | Val Loss: 0.8802 | Val F1: 0.8005 | Time:
73.67s

Best F1-Macro in Fold 4: 0.8005
Fold total time: 294.45s | Mean time/epoch: 73.61s

================ BERT FOLD 5/5 ================

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
```

```
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
```

```
Epoch 1/4 | Train Loss: 0.5909 | Val Loss: 0.5353 | Val F1: 0.7411 | Time:
73.77s
Epoch 2/4 | Train Loss: 0.4128 | Val Loss: 0.5203 | Val F1: 0.7677 | Time:
73.60s
Epoch 3/4 | Train Loss: 0.2511 | Val Loss: 0.7445 | Val F1: 0.7658 | Time:
73.44s
Epoch 4/4 | Train Loss: 0.1441 | Val Loss: 0.9659 | Val F1: 0.7676 | Time:
73.48s

Best F1-Macro in Fold 5: 0.7677
Fold total time: 294.30s | Mean time/epoch: 73.57s

========================================
Mean F1-Macro for BERT (5 Folds): 0.7859
========================================
```

1.4.2 DistillBERT

[19]:
```
MODEL_NAME = "distilbert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

[20]:
```
# Preparing raw texts
X_train_distilbert = df_train['tweet']
y_train_distilbert = df_train['label']

X_test_distilbert = df_val['tweet']
y_test_distilbert = df_val['label']
```

[21]:
```
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
from torch.utils.data import Subset

torch.manual_seed(SEED)
np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Training DistilBERT in: {device}\n")
```

```
EPOCHS_DISTILBERT = 4
BATCH_SIZE = 16
N_SPLITS = 5

skf_distilbert = StratifiedKFold(n_splits=N_SPLITS, shuffle=True,␣
↪random_state=SEED)
y_train_array_distilbert = y_train_distilbert.values
distilbert_fold_metrics = []
best_distilbert_global_f1 = 0
best_distilbert_model_state = None

distilbert_full_dataset = BERTDataset(X_train_distilbert, y_train_distilbert,␣
↪tokenizer)

for fold, (train_idx, val_idx) in enumerate(skf_distilbert.split(np.
↪zeros(len(y_train_array_distilbert)), y_train_array_distilbert)):
    print(f"================ DistilBERT FOLD {fold + 1}/{N_SPLITS}␣
↪================")

    train_sub = Subset(distilbert_full_dataset, train_idx)
    val_sub = Subset(distilbert_full_dataset, val_idx)

    train_loader = DataLoader(train_sub, batch_size=BATCH_SIZE, shuffle=True)
    val_loader = DataLoader(val_sub, batch_size=BATCH_SIZE, shuffle=False)

    model = BERTClassifier(MODEL_NAME, num_classes=2, freeze_bert=False).
↪to(device)

    # Print parameter count once (first fold only)
    if fold == 0:
        distilbert_total_params = sum(p.numel() for p in model.parameters())
        distilbert_trainable_params = sum(p.numel() for p in model.parameters()␣
↪if p.requires_grad)
        print(f"\nModel architecture:")
        print(model)
        print(f"\nTotal parameters: {total_params:,}")
        print(f"Trainable parameters: {trainable_params:,}\n")

    optimizer = torch.optim.AdamW([
        {'params': model.bert.parameters(), 'lr': 2e-5},
        {'params': model.classifier.parameters(), 'lr': 1e-4}
    ])

    total_steps = len(train_loader) * EPOCHS_DISTILBERT
    warmup_steps = int(total_steps * 0.1)
```

```
    scheduler = get_linear_schedule_with_warmup(optimizer,␣
↪num_warmup_steps=warmup_steps, num_training_steps=total_steps)

    criterion = nn.CrossEntropyLoss()
    best_fold_f1 = 0
    epoch_times = []
    fold_start = perf_counter()

    for epoch in range(EPOCHS_DISTILBERT):
        epoch_start = perf_counter()

        model.train()
        train_loss = 0
        all_train_preds, all_train_labels = [], []

        for batch in train_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['label'].to(device)

            optimizer.zero_grad()
            logits = model(input_ids, attention_mask)
            loss = criterion(logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            scheduler.step()

            train_loss += loss.item()
            _, predicted = torch.max(logits, 1)
            all_train_preds.extend(predicted.cpu().numpy())
            all_train_labels.extend(labels.cpu().numpy())

        model.eval()
        val_loss = 0
        all_val_preds, all_val_labels = [], []

        with torch.no_grad():
            for batch in val_loader:
                input_ids = batch['input_ids'].to(device)
                attention_mask = batch['attention_mask'].to(device)
                labels = batch['label'].to(device)

                logits = model(input_ids, attention_mask)
                loss = criterion(logits, labels)

                val_loss += loss.item()
```

```
                _, predicted = torch.max(logits, 1)
                all_val_preds.extend(predicted.cpu().numpy())
                all_val_labels.extend(labels.cpu().numpy())

        epoch_time = perf_counter() - epoch_start
        epoch_times.append(epoch_time)

        val_f1 = f1_score(all_val_labels, all_val_preds, average='macro')
        print(f"Epoch {epoch+1}/{EPOCHS_DISTILBERT} | "
              f"Train Loss: {train_loss/len(train_loader):.4f} | "
              f"Val Loss: {val_loss/len(val_loader):.4f} | "
              f"Val F1: {val_f1:.4f} | "
              f"Time: {epoch_time:.2f}s")

        if val_f1 > best_fold_f1:
            best_fold_f1 = val_f1
            best_model_state_for_this_fold = model.state_dict().copy()

    fold_time = perf_counter() - fold_start
    distilbert_mean_epoch_time = sum(epoch_times) / len(epoch_times)

    distilbert_fold_metrics.append(best_fold_f1)
    print(f"\nMejor F1-Macro en Fold {fold+1}: {best_fold_f1:.4f}")
    print(f"Fold total time: {fold_time:.2f}s | Mean time/epoch:␣
 ↪{distilbert_mean_epoch_time:.2f}s\n")

    if best_fold_f1 > best_distilbert_global_f1:
        best_distilbert_global_f1 = best_fold_f1
        best_distilbert_model_state = best_model_state_for_this_fold

print("=" * 40)
print(f"F1-Macro Promedio DistilBERT (5 Folds): {np.
 ↪mean(distilbert_fold_metrics):.4f}")
print("=" * 40)
```

```
Training DistilBERT in: cuda

================ DistilBERT FOLD 1/5 ================

Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]

DistilBertModel LOAD REPORT from: distilbert-base-multilingual-cased
Key                      | Status     |  |
-------------------------+------------+--+-
vocab_transform.weight   | UNEXPECTED |  |
vocab_layer_norm.weight  | UNEXPECTED |  |
vocab_transform.bias     | UNEXPECTED |  |
vocab_projector.bias     | UNEXPECTED |  |
```

```
vocab_layer_norm.bias   | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.


Model architecture:
BERTClassifier(
  (bert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(119547, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12,
elementwise_affine=True)
        )
      )
    )
  )
  (dropout): Dropout(p=0.3, inplace=False)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
)

Total parameters: 3,331,562
Trainable parameters: 3,331,562

Epoch 1/4 | Train Loss: 0.6031 | Val Loss: 0.5031 | Val F1: 0.7612 | Time:
44.20s
```

```
Epoch 2/4 | Train Loss: 0.4346 | Val Loss: 0.4737 | Val F1: 0.7761 | Time:
50.13s
Epoch 3/4 | Train Loss: 0.2851 | Val Loss: 0.5624 | Val F1: 0.7753 | Time:
50.03s
Epoch 4/4 | Train Loss: 0.1681 | Val Loss: 0.7427 | Val F1: 0.7726 | Time:
49.94s

Mejor F1-Macro en Fold 1: 0.7761
Fold total time: 194.33s | Mean time/epoch: 48.58s

================ DistilBERT FOLD 2/5 ================

Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]

DistilBertModel LOAD REPORT from: distilbert-base-multilingual-cased
Key                      | Status     |  |
-------------------------+------------+--+-
vocab_transform.weight   | UNEXPECTED |  |
vocab_layer_norm.weight  | UNEXPECTED |  |
vocab_transform.bias     | UNEXPECTED |  |
vocab_projector.bias     | UNEXPECTED |  |
vocab_layer_norm.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.6114 | Val Loss: 0.5352 | Val F1: 0.7127 | Time:
49.92s
Epoch 2/4 | Train Loss: 0.4307 | Val Loss: 0.4989 | Val F1: 0.7634 | Time:
49.94s
Epoch 3/4 | Train Loss: 0.2833 | Val Loss: 0.5516 | Val F1: 0.7567 | Time:
49.97s
Epoch 4/4 | Train Loss: 0.1705 | Val Loss: 0.7442 | Val F1: 0.7624 | Time:
49.96s

Mejor F1-Macro en Fold 2: 0.7634
Fold total time: 199.81s | Mean time/epoch: 49.95s

================ DistilBERT FOLD 3/5 ================

Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]

DistilBertModel LOAD REPORT from: distilbert-base-multilingual-cased
Key                      | Status     |  |
-------------------------+------------+--+-
vocab_transform.weight   | UNEXPECTED |  |
vocab_layer_norm.weight  | UNEXPECTED |  |
vocab_transform.bias     | UNEXPECTED |  |
vocab_projector.bias     | UNEXPECTED |  |
```

```
vocab_layer_norm.bias   | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.6186 | Val Loss: 0.5199 | Val F1: 0.7428 | Time:
50.01s
Epoch 2/4 | Train Loss: 0.4552 | Val Loss: 0.4853 | Val F1: 0.7750 | Time:
50.09s
Epoch 3/4 | Train Loss: 0.3150 | Val Loss: 0.5446 | Val F1: 0.7671 | Time:
50.03s
Epoch 4/4 | Train Loss: 0.2174 | Val Loss: 0.6400 | Val F1: 0.7668 | Time:
50.01s

Mejor F1-Macro en Fold 3: 0.7750
Fold total time: 200.15s | Mean time/epoch: 50.03s

================ DistilBERT FOLD 4/5 ================

Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]

DistilBertModel LOAD REPORT from: distilbert-base-multilingual-cased
Key                      | Status     |  |
-------------------------+------------+--+-
vocab_transform.weight   | UNEXPECTED |  |
vocab_layer_norm.weight  | UNEXPECTED |  |
vocab_transform.bias     | UNEXPECTED |  |
vocab_projector.bias     | UNEXPECTED |  |
vocab_layer_norm.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.6035 | Val Loss: 0.4970 | Val F1: 0.7512 | Time:
50.09s
Epoch 2/4 | Train Loss: 0.4444 | Val Loss: 0.5035 | Val F1: 0.7755 | Time:
50.04s
Epoch 3/4 | Train Loss: 0.2946 | Val Loss: 0.6022 | Val F1: 0.7530 | Time:
50.02s
Epoch 4/4 | Train Loss: 0.1894 | Val Loss: 0.7419 | Val F1: 0.7718 | Time:
50.05s

Mejor F1-Macro en Fold 4: 0.7755
Fold total time: 200.22s | Mean time/epoch: 50.05s

================ DistilBERT FOLD 5/5 ================

Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]
```

```
DistilBertModel LOAD REPORT from: distilbert-base-multilingual-cased
Key                      | Status     |  |
-------------------------+------------+--+-
vocab_transform.weight   | UNEXPECTED |  |
vocab_layer_norm.weight  | UNEXPECTED |  |
vocab_transform.bias     | UNEXPECTED |  |
vocab_projector.bias     | UNEXPECTED |  |
vocab_layer_norm.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Epoch 1/4 | Train Loss: 0.6091 | Val Loss: 0.5530 | Val F1: 0.7321 | Time:
50.03s
Epoch 2/4 | Train Loss: 0.4323 | Val Loss: 0.5528 | Val F1: 0.7439 | Time:
50.06s
Epoch 3/4 | Train Loss: 0.2887 | Val Loss: 0.6199 | Val F1: 0.7520 | Time:
50.01s
Epoch 4/4 | Train Loss: 0.1775 | Val Loss: 0.7920 | Val F1: 0.7449 | Time:
49.97s

Mejor F1-Macro en Fold 5: 0.7520
Fold total time: 200.09s | Mean time/epoch: 50.02s

========================================
F1-Macro Promedio DistilBERT (5 Folds): 0.7684
========================================
```

1.4.3 Evaluating best Transformer

[22]:
```python
print("Evaluating BERT in TEST set...")

MODEL_NAME = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
bert_test_dataset = BERTDataset(X_test_bert, y_test_bert, tokenizer)
bert_test_loader = DataLoader(bert_test_dataset, batch_size=BATCH_SIZE,␣
 ↪shuffle=False)

final_bert = BERTClassifier(MODEL_NAME, num_classes=2).to(device)
final_bert.load_state_dict(best_bert_model_state)
final_bert.eval()

bert_test_preds = []
bert_test_labels = []

from time import time
start_time = time()
```

```python
with torch.no_grad():
    for batch in bert_test_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['label'].to(device)

        logits = final_bert(input_ids, attention_mask)
        _, predicted = torch.max(logits, 1)

        bert_test_preds.extend(predicted.cpu().numpy())
        bert_test_labels.extend(labels.cpu().numpy())

bert_inf_time = time() - start_time

bert_test_f1_saved = f1_score(bert_test_labels, bert_test_preds,␣
 ↪average='macro')

print(f"\nFinal F1-Macro BERT (Test Set): {bert_test_f1_saved:.4f}")
print(f"Total Inference Time: {bert_inf_time:.3f} seconds")
print("\nClasification Report:\n", classification_report(bert_test_labels,␣
 ↪bert_test_preds))
```

```
Evaluating BERT in TEST set…

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.


Final F1-Macro BERT (Test Set): 0.7907
Total Inference Time: 4.251 seconds

Clasification Report:
               precision    recall  f1-score   support
```

```
           0       0.77      0.78      0.78       479
           1       0.81      0.81      0.81       559

    accuracy                           0.79      1038
   macro avg       0.79      0.79      0.79      1038
weighted avg       0.79      0.79      0.79      1038
```

1.5 Comparative Analysis and Results

In this final stretch of the notebook, we gather all the obtained results to establish a comprehensive comparison between:

1. **Baseline (Assignment 1):** TF-IDF + Logistic Regression
2. **Recurrent Model:** BiLSTM with Optuna-tuned hyperparameters
3. **Transformer Model:** BERT-base-uncased with full fine-tuning

To provide a holistic answer to our initial research questions, we evaluate the performance of these models considering three fundamental dimensions. First, we analyze pure predictive performance based on the F1-Macro metric on the isolated test set, ensuring a fair evaluation that penalizes class imbalance. Second, we examine the structural complexity of each approach by counting the total number of trainable parameters. Finally, we weigh the associated computational cost, meticulously recording both the training time required per epoch and the latency experienced during the inference process on new data.

[23]:
```
print(f'BiLSTM Test F1-Macro:    {bilstm_test_f1:.4f}')
print(f'BiLSTM Inference Time:   {bilstm_inf_time:.3f}s')
print(f'BiLSTM predictions saved ({len(bilstm_test_preds)} samples)')
print(f'BiLSTM mean training time per epoch: {bilstm_mean_epoch_time:.3f}␣
↪seconds')
print(f'BiLSTM Total Parameters: {bilstm_total_params}')
print(f'BiLSTM Total Trainable Parameters: {bilstm_trainable_params}')
```

```
BiLSTM Test F1-Macro:    0.7177
BiLSTM Inference Time:   0.556s
BiLSTM predictions saved (1038 samples)
BiLSTM mean training time per epoch: 1.417 seconds
BiLSTM Total Parameters: 3331562
BiLSTM Total Trainable Parameters: 3331562
```

[24]:
```
print(f'BERT Test F1-Macro:    {bert_test_f1_saved:.4f}')
print(f'BERT Inference Time:   {bert_inf_time:.3f}s')
print(f'BERT predictions saved ({len(bert_test_preds)} samples)')
print(f'BERT mean training time per epoch: {bert_mean_epoch_time:.3f} seconds')
print(f'BERT Total Parameters: {bert_total_params}')
print(f'BERT Total Trainable Parameters: {bert_trainable_params}')
```

```
BERT Test F1-Macro:    0.7907
```

```
BERT Inference Time:   4.251s
BERT predictions saved (1038 samples)
BERT mean training time per epoch: 73.570 seconds
BERT Total Parameters: 177854978
BERT Total Trainable Parameters: 177854978
```

[25]:
```python
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import numpy as np
import pandas as pd
from sklearn.metrics import (f1_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay,
                             precision_score, recall_score)

#
#  Baseline values (from Assignment 1)
#
baseline_test_f1 = 0.75
baseline_test_acc = 0.75
baseline_params = 'N/A'
baseline_train_time = 'N/A'
baseline_inf_time = 'N/A'

#
#  Build comparison DataFrame
#
comparison = pd.DataFrame({
    'Model': [
        'TF-IDF + LogReg (Baseline)',
        'BiLSTM (Optuna-tuned)',
        'BERT-base-multilingual-cased (Fine-tuned)'
    ],
    'Test F1-Macro': [
        f'{baseline_test_f1:.4f}',
        f'{bilstm_test_f1:.4f}',
        f'{bert_test_f1_saved:.4f}'
    ],
    'Parameters': [
        baseline_params,
        f'{bilstm_total_params:,}',
        f'{bert_total_params:,}'
    ],
    'Training Time per Epoch': [
        baseline_train_time,
        f'{bilstm_mean_epoch_time:.3f}s',
        f'{bert_mean_epoch_time:.3f}s'
    ],
```

```
    'Inference (test)': [
        baseline_inf_time,
        f'{bilstm_inf_time:.3f}s',
        f'{bert_inf_time:.3f}s'
    ],
})

print('=' * 80)
print('     MODEL COMPARISON - EXIST 2025 Task 1.1 (Sexism Identification)')
print('=' * 80)
display(comparison.style.set_properties(**{'text-align': 'center'}).
 ↪hide(axis='index'))
```

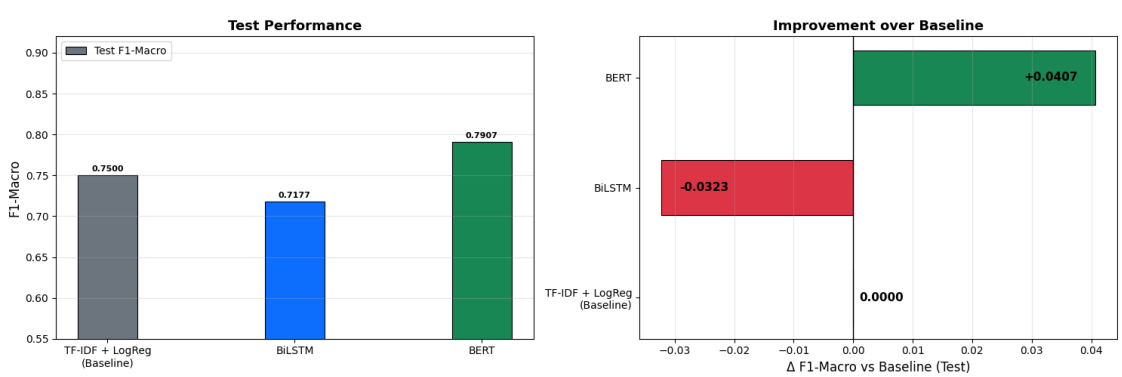
```
================================================================================
     MODEL COMPARISON - EXIST 2025 Task 1.1 (Sexism Identification)
================================================================================
```

```
<pandas.io.formats.style.Styler at 0x78068e4a70e0>
```

[26]:

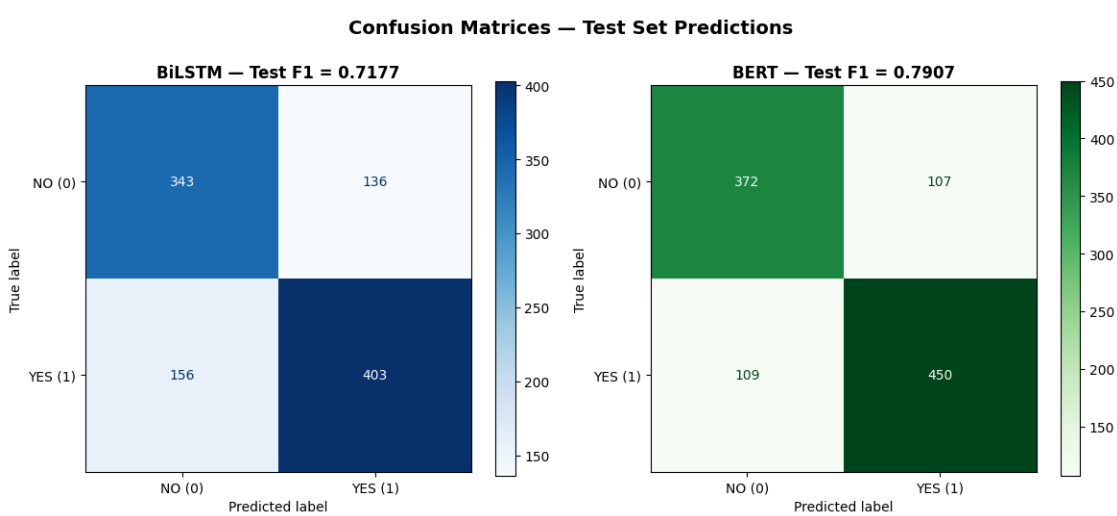
```
#
#  Visualization 1: Grouped bar chart - CV F1 vs Test F1
#
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

models  = ['TF-IDF + LogReg\n(Baseline)', 'BiLSTM', 'BERT']
tst_f1s = [baseline_test_f1, bilstm_test_f1, bert_test_f1_saved]
colors  = ['#6C757D', '#0D6EFD', '#198754']

# --- Grouped bar chart ---
x = np.arange(len(models))
width = 0.32

bars = axes[0].bar(x, tst_f1s, width, label='Test F1-Macro',
                      color=colors, alpha=1.0, edgecolor='black', linewidth=0.8)

axes[0].set_ylabel('F1-Macro', fontsize=12)
axes[0].set_title('Test Performance', fontsize=13, fontweight='bold')
axes[0].set_xticks(x)
axes[0].set_xticklabels(models, fontsize=10)
axes[0].set_ylim(0.55, 0.92)
axes[0].legend(fontsize=10, loc='upper left')
axes[0].yaxis.set_major_formatter(mticker.FormatStrFormatter('%.2f'))
axes[0].grid(axis='y', alpha=0.3)

for bar in bars:
    axes[0].annotate(f'{bar.get_height():.4f}',
                      xy=(bar.get_x() + bar.get_width() / 2, bar.get_height()),
                      xytext=(0, 4), textcoords='offset points',
```

```
                    ha='center', fontsize=8, fontweight='bold')

# --- Delta plot (improvement over baseline) ---
deltas_tst = [0,
              bilstm_test_f1 - baseline_test_f1,
              bert_test_f1_saved - baseline_test_f1]

bar_colors_delta = ['#6C757D' if d == 0 else ('#198754' if d > 0 else␣
 ↪'#DC3545') for d in deltas_tst]
axes[1].barh(models, deltas_tst, color=bar_colors_delta, edgecolor='black',
             height=0.5, linewidth=0.8)
axes[1].axvline(x=0, color='black', linewidth=1)
axes[1].set_xlabel('Δ F1-Macro vs Baseline (Test)', fontsize=12)
axes[1].set_title('Improvement over Baseline', fontsize=13, fontweight='bold')
axes[1].grid(axis='x', alpha=0.3)

for i, v in enumerate(deltas_tst):
    sign = '+' if v > 0 else ''
    if v == 0:
        axes[1].text(0.001, i, '0.0000', va='center', ha='left',␣
 ↪fontweight='bold', fontsize=11)
    else:
        # place label inside the bar, near the end
        offset = -0.003 if v > 0 else 0.003
        ha = 'right' if v > 0 else 'left'
        axes[1].text(v + offset, i, f'{sign}{v:.4f}',
                     va='center', ha=ha, fontweight='bold', fontsize=11)

plt.tight_layout()
plt.savefig('../results/model_comparison_barplot.png', dpi=150,␣
 ↪bbox_inches='tight')
plt.show()
print('Figure saved → results/model_comparison_barplot.png')
```



```
Figure saved → results/model_comparison_barplot.png
```

[27]:
```
#
#  Visualization 2: Side-by-side Confusion Matrices (Test Set)
#

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# --- BiLSTM ---
cm_bilstm = confusion_matrix(bilstm_test_labels, bilstm_test_preds)
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm_bilstm, display_labels=['NO␣
↪(0)', 'YES (1)'])
disp1.plot(ax=axes[0], cmap='Blues', values_format='d')
axes[0].set_title(f'BiLSTM - Test F1 = {bilstm_test_f1:.4f}',
                  fontsize=12, fontweight='bold')

# --- BERT ---
cm_bert = confusion_matrix(bert_test_labels, bert_test_preds)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_bert, display_labels=['NO␣
↪(0)', 'YES (1)'])
disp2.plot(ax=axes[1], cmap='Greens', values_format='d')
axes[1].set_title(f'BERT - Test F1 = {bert_test_f1_saved:.4f}',
                  fontsize=12, fontweight='bold')

fig.suptitle('Confusion Matrices - Test Set Predictions',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('../results/confusion_matrices_comparison.png', dpi=150,␣
↪bbox_inches='tight')
plt.show()
print('Figure saved → results/confusion_matrices_comparison.png')
```



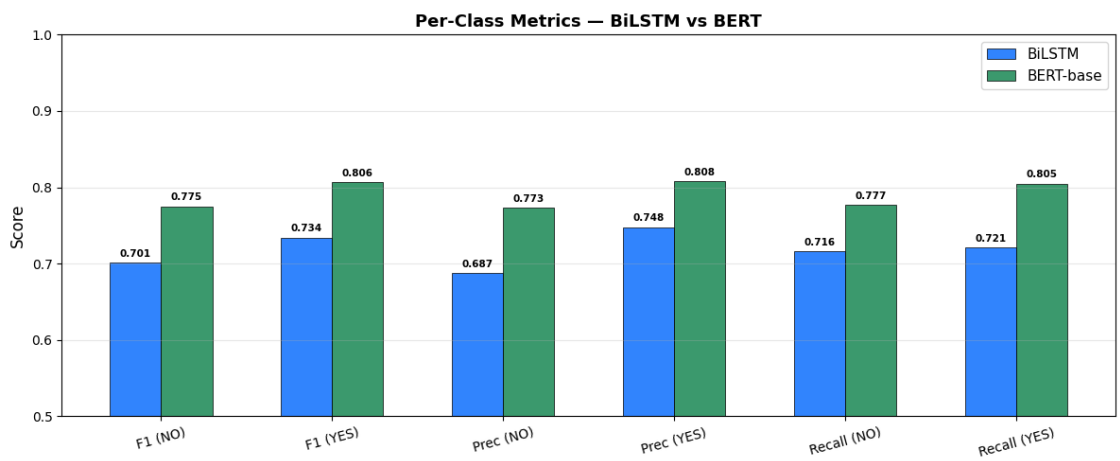
```
Figure saved → results/confusion_matrices_comparison.png
```

[28]:
```python
#
#  Per-class F1 / Precision / Recall comparison
#

def get_per_class_metrics(y_true, y_pred, name):
    return {
        'Model': name,
        'F1 (NO)':        round(f1_score(y_true, y_pred, pos_label=0), 4),
        'F1 (YES)':       round(f1_score(y_true, y_pred, pos_label=1), 4),
        'Prec (NO)':      round(precision_score(y_true, y_pred, pos_label=0),␣
 ↪4),
        'Prec (YES)':     round(precision_score(y_true, y_pred, pos_label=1),␣
 ↪4),
        'Recall (NO)':    round(recall_score(y_true, y_pred, pos_label=0), 4),
        'Recall (YES)':   round(recall_score(y_true, y_pred, pos_label=1), 4),
        'F1-Macro':       round(f1_score(y_true, y_pred, average='macro'), 4),
    }

rows = [
    get_per_class_metrics(bilstm_test_labels, bilstm_test_preds, 'BiLSTM'),
    get_per_class_metrics(bert_test_labels, bert_test_preds, 'BERT-base'),
]

df_perclass = pd.DataFrame(rows)

print('=' * 80)
print('     PER-CLASS METRICS ON TEST SET')
print('=' * 80)
display(df_perclass.style.set_properties(**{'text-align': 'center'}).
 ↪hide(axis='index'))

#   Grouped bar chart
fig, ax = plt.subplots(figsize=(12, 5))
metrics_names = ['F1 (NO)', 'F1 (YES)', 'Prec (NO)', 'Prec (YES)', 'Recall␣
 ↪(NO)', 'Recall (YES)']
x = np.arange(len(metrics_names))
width = 0.30

for i, row in enumerate(rows):
    vals = [row[k] for k in metrics_names]
    color = ['#0D6EFD', '#198754'][i]
    ax.bar(x + i * width, vals, width, label=row['Model'],
           color=color, alpha=0.85, edgecolor='black', linewidth=0.6)
    for j, v in enumerate(vals):
```

```
        ax.text(x[j] + i * width, v + 0.008, f'{v:.3f}',
                ha='center', fontsize=7.5, fontweight='bold')

ax.set_xticks(x + width / 2)
ax.set_xticklabels(metrics_names, fontsize=10, rotation=15)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Per-Class Metrics - BiLSTM vs BERT', fontsize=13,␣
 ↪fontweight='bold')
ax.legend(fontsize=11)
ax.set_ylim(0.5, 1.0)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('../results/per_class_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print('Figure saved → results/per_class_comparison.png')
```

```
================================================================================
     PER-CLASS METRICS ON TEST SET
================================================================================
```

```
<pandas.io.formats.style.Styler at 0x7802ad5fd590>
```




```
Figure saved → results/per_class_comparison.png
```

[29]:
```
#
#  Model Complexity & Efficiency Summary
#

complexity_df = pd.DataFrame({
    'Model':            ['BiLSTM', 'BERT-base-uncased'],
    'Total Params':     [f'{bilstm_total_params:,}', f'{bert_total_params:,}'],
```

```
    'Trainable Params': [f'{bilstm_trainable_params:,}',␣
 ↪f'{bert_trainable_params:,}'],
    'Test F1-Macro':    [f'{bilstm_test_f1:.4f}', f'{bert_test_f1_saved:.4f}'],
    'Inference (s)':    [f'{bilstm_inf_time:.3f}', f'{bert_inf_time:.3f}'],
    'F1 / M params':    [f'{bilstm_test_f1/(bilstm_total_params/1e6):.4f}',
                         f'{bert_test_f1_saved/(bert_total_params/1e6):.4f}'],
})

print(' ' + ' ' * 64 + ' ')
print('   MODEL COMPLEXITY & EFFICIENCY SUMMARY' + ' ' * 25 + ' ')
print(' ' + ' ' * 64 + ' ')
display(complexity_df.style.set_properties(**{'text-align': 'center'}).
 ↪hide(axis='index'))

ratio = bert_total_params / bilstm_total_params
print(f'\n→ BERT has {ratio:.1f}× more parameters than BiLSTM')
print(f'→ BERT inference is {bert_inf_time/bilstm_inf_time:.1f}× slower than␣
 ↪BiLSTM')
```

```
  MODEL COMPLEXITY & EFFICIENCY SUMMARY
```

```
<pandas.io.formats.style.Styler at 0x7802ad69d1d0>
```

```
→ BERT has 53.4× more parameters than BiLSTM
→ BERT inference is 7.6× slower than BiLSTM
```

1.6 Learning Curve Analysis

How much data do neural models need to surpass the classical baseline?

We train both the **BiLSTM** (with best Optuna hyperparameters) and **BERT-base** on **25%, 50%, 75%, and 100%** of the training data, evaluating each on the held-out test set. The baseline TF-IDF + Logistic Regression result (F1-Macro = 0.75) serves as a constant reference line.

This experiment answers two key questions: 1. **At what training size do neural models surpass the baseline?** 2. **Does the Transformer benefit more from additional data than the LSTM?**

[30]:
```
#
#  Learning Curve - BiLSTM
#
#
# Uses the best hyperparameters from Optuna (stored in `best_params`
# or `study.best_trial.params`).  For each data fraction, we train
# on a stratified subset and evaluate on the FULL test set.
```

```
#
#

from sklearn.model_selection import StratifiedShuffleSplit
from time import time

DATA_FRACTIONS = [0.25, 0.50, 0.75, 1.00]
EPOCHS_LC = 20  # Same as main training
BATCH_SIZE = 512 # Same as main training

bp = study.best_trial.params  # best hyperparams

bilstm_lc_results = []  # List of dicts: {fraction, n_samples, test_f1,␣
↪train_time}

for frac in DATA_FRACTIONS:
    print(f'\n{"="*60}')
    print(f'  BiLSTM - Training on {frac*100:.0f}% of data')
    print(f'{"="*60}')

    #   Stratified subset
    if frac < 1.0:
        sss = StratifiedShuffleSplit(n_splits=1, train_size=frac,␣
↪random_state=SEED)
        sub_idx, _ = next(sss.split(np.zeros(len(df_train)), df_train['label'].
↪values))
    else:
        sub_idx = np.arange(len(df_train))

    n_samples = len(sub_idx)
    print(f'  Samples: {n_samples}')

    #   DataLoader
    sub_dataset = Subset(train_dataset, sub_idx)
    sub_loader  = DataLoader(sub_dataset, BATCH_SIZE, shuffle=True)

    #   Model
    model_lc = BiLSTMClassifier(
        vocab_size    = VOCAB_SIZE,
        embedding_dim = bp['embedding_dim'],
        hidden_dim    = bp['hidden_dim'],
        output_dim    = OUTPUT_DIM,
        n_layers      = bp['n_layers'],
        dropout       = bp['dropout'],
    ).to(device)

    criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model_lc.parameters(), lr=bp['lr'])
early_stop = EarlyStopping(patience=5)

#   Train
t0 = time()
for epoch in range(EPOCHS_LC):
    model_lc.train()
    for texts, labels, lengths in sub_loader:
        texts, labels = texts.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model_lc(texts, lengths), labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model_lc.parameters(), max_norm=1.0)
        optimizer.step()

    # Quick val on test set each epoch for early stopping
    model_lc.eval()
    preds, targets = [], []
    with torch.no_grad():
        for texts, labels, lengths in test_loader_real:
            texts = texts.to(device)
            out = model_lc(texts, lengths)
            preds.extend(out.argmax(1).cpu().numpy())
            targets.extend(labels.numpy())

    ep_f1 = f1_score(targets, preds, average='macro')
    if early_stop(ep_f1, model_lc):
        print(f'  Early stop at epoch {epoch+1}')
        break

train_time = time() - t0

#   Final evaluation with best model state
model_lc.load_state_dict(early_stop.best_model_state)
model_lc.eval()
preds_final, targets_final = [], []
with torch.no_grad():
    for texts, labels, lengths in test_loader_real:
        texts = texts.to(device)
        out = model_lc(texts, lengths)
        preds_final.extend(out.argmax(1).cpu().numpy())
        targets_final.extend(labels.numpy())

test_f1_lc = f1_score(targets_final, preds_final, average='macro')

bilstm_lc_results.append({
    'fraction':   frac,
```

```
        'n_samples':  n_samples,
        'test_f1':    test_f1_lc,
        'train_time': train_time,
    })

    print(f'  Test F1-Macro: {test_f1_lc:.4f} | Train time: {train_time:.1f}s')

print('\n' + '='*60)
print('BiLSTM Learning Curve - Summary')
print('='*60)
for r in bilstm_lc_results:
    print(f"  {r['fraction']*100:5.0f}% ({r['n_samples']:5d} samples) -> F1:␣
 ↪{r['test_f1']:.4f} | Time: {r['train_time']:.1f}s")
```

```
============================================================
  BiLSTM - Training on 25% of data
============================================================
  Samples: 1730
  Test F1-Macro: 0.6371 | Train time: 15.6s

============================================================
  BiLSTM - Training on 50% of data
============================================================
  Samples: 3460
  Early stop at epoch 16
  Test F1-Macro: 0.6847 | Train time: 17.3s

============================================================
  BiLSTM - Training on 75% of data
============================================================
  Samples: 5190
  Early stop at epoch 12
  Test F1-Macro: 0.7201 | Train time: 16.7s

============================================================
  BiLSTM - Training on 100% of data
============================================================
  Samples: 6920
  Early stop at epoch 13
  Test F1-Macro: 0.7067 | Train time: 22.2s

============================================================
BiLSTM Learning Curve - Summary
============================================================
     25% ( 1730 samples) -> F1: 0.6371 | Time: 15.6s
     50% ( 3460 samples) -> F1: 0.6847 | Time: 17.3s
     75% ( 5190 samples) -> F1: 0.7201 | Time: 16.7s
```

```
100% ( 6920 samples) -> F1: 0.7067 | Time: 22.2s
```

[31]:
```
#
#  Learning Curve -- BERT-base-multilingual-cased (Fine-tuning from pre-trained)
#
#
# For each data fraction we FINE-TUNE from the pre-trained checkpoint
# (NOT training from scratch). The model loads pre-trained weights via
# AutoModel.from_pretrained() and then adapts them to our task.
#
# Same training config as the main BERT training loop:
#   - Differential learning rates (2e-5 for BERT, 1e-4 for classifier)
#   - Linear warmup scheduler
#   - Gradient clipping
#
#

from transformers import AutoModel, AutoTokenizer,␣
↪get_linear_schedule_with_warmup

EPOCHS_BERT_LC = 4
BATCH_SIZE_BERT_LC = 16

bert_lc_results = []  # List of dicts

for frac in DATA_FRACTIONS:
    print(f'\n{"="*60}')
    print(f'  BERT (fine-tuning) -- Training on {frac*100:.0f}% of data')
    print(f'{"="*60}')

    #   Stratified subset
    if frac < 1.0:
        sss = StratifiedShuffleSplit(n_splits=1, train_size=frac,␣
↪random_state=SEED)
        sub_idx, _ = next(sss.split(np.zeros(len(df_train)), df_train['label'].
↪values))
    else:
        sub_idx = np.arange(len(df_train))

    n_samples = len(sub_idx)
    print(f'  Samples: {n_samples}')

    #   Dataset & DataLoader
    bert_sub_dataset = BERTDataset(
        df_train['tweet'].iloc[sub_idx].reset_index(drop=True),
        df_train['label'].iloc[sub_idx].reset_index(drop=True),
        tokenizer
```

```
    )
    bert_sub_loader = DataLoader(bert_sub_dataset,␣
↪batch_size=BATCH_SIZE_BERT_LC, shuffle=True)

    #   Load PRE-TRAINED model (fine-tuning, NOT from scratch)
    # This loads the full bert-base-uncased pre-trained weights
    # and adds a fresh classification head on top.
    model_bert_lc = BERTClassifier(MODEL_NAME, num_classes=2,␣
↪freeze_bert=False).to(device)

    # Differential learning rates: low for pre-trained BERT layers,
    # higher for the randomly-initialized classifier head
    optimizer_bert = torch.optim.AdamW([
        {'params': model_bert_lc.bert.parameters(), 'lr': 2e-5},
        {'params': model_bert_lc.classifier.parameters(), 'lr': 1e-4}
    ])

    total_steps = len(bert_sub_loader) * EPOCHS_BERT_LC
    warmup_steps = int(total_steps * 0.1)
    scheduler_lc = get_linear_schedule_with_warmup(
        optimizer_bert, num_warmup_steps=warmup_steps,␣
↪num_training_steps=total_steps
    )
    criterion_bert = nn.CrossEntropyLoss()

    best_f1_this_frac = 0
    best_state_this_frac = None

    #   Fine-tune
    t0 = time()
    for epoch in range(EPOCHS_BERT_LC):
        model_bert_lc.train()
        for batch in bert_sub_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['label'].to(device)

            optimizer_bert.zero_grad()
            logits = model_bert_lc(input_ids, attention_mask)
            loss = criterion_bert(logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model_bert_lc.parameters(),␣
↪max_norm=1.0)
            optimizer_bert.step()
            scheduler_lc.step()

        #   Evaluate on test set
```

```
        model_bert_lc.eval()
        preds_b, targets_b = [], []
        with torch.no_grad():
            for batch in bert_test_loader:
                input_ids = batch['input_ids'].to(device)
                attention_mask = batch['attention_mask'].to(device)
                labels_b = batch['label']

                logits = model_bert_lc(input_ids, attention_mask)
                _, predicted = torch.max(logits, 1)
                preds_b.extend(predicted.cpu().numpy())
                targets_b.extend(labels_b.numpy())

        ep_f1 = f1_score(targets_b, preds_b, average='macro')
        print(f'  Epoch {epoch+1}/{EPOCHS_BERT_LC} | Test F1: {ep_f1:.4f}')

        if ep_f1 > best_f1_this_frac:
            best_f1_this_frac = ep_f1
            best_state_this_frac = {k: v.clone() for k, v in model_bert_lc.
 ↪state_dict().items()}

    train_time = time() - t0

    bert_lc_results.append({
        'fraction':   frac,
        'n_samples':  n_samples,
        'test_f1':    best_f1_this_frac,
        'train_time': train_time,
    })

    print(f'  Best Test F1-Macro: {best_f1_this_frac:.4f} | Train time:␣
 ↪{train_time:.1f}s')

    # Free GPU memory
    del model_bert_lc, optimizer_bert, scheduler_lc
    torch.cuda.empty_cache()

print('\n' + '='*60)
print('BERT Learning Curve (Fine-tuned) -- Summary')
print('='*60)
for r in bert_lc_results:
    print(f"  {r['fraction']*100:5.0f}% ({r['n_samples']:5d} samples) -> F1:␣
 ↪{r['test_f1']:.4f} | Time: {r['train_time']:.1f}s")
```

```
============================================================
  BERT (fine-tuning) -- Training on 25% of data
```

```
============================================================
  Samples: 1730

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

  Epoch 1/4 | Test F1: 0.6846
  Epoch 2/4 | Test F1: 0.7474
  Epoch 3/4 | Test F1: 0.7535
  Epoch 4/4 | Test F1: 0.7482
  Best Test F1-Macro: 0.7535 | Train time: 109.6s

============================================================
  BERT (fine-tuning) -- Training on 50% of data
============================================================
  Samples: 3460

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

  Epoch 1/4 | Test F1: 0.6835
  Epoch 2/4 | Test F1: 0.7676
  Epoch 3/4 | Test F1: 0.7574
```

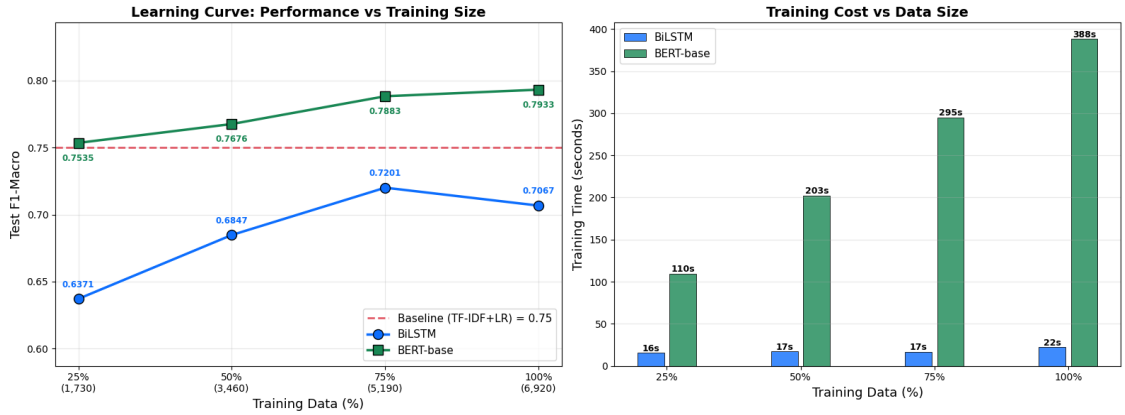
```
  Epoch 4/4 | Test F1: 0.7595
  Best Test F1-Macro: 0.7676 | Train time: 202.5s

============================================================
  BERT (fine-tuning) -- Training on 75% of data
============================================================
  Samples: 5190

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

  Epoch 1/4 | Test F1: 0.7720
  Epoch 2/4 | Test F1: 0.7883
  Epoch 3/4 | Test F1: 0.7822
  Epoch 4/4 | Test F1: 0.7881
  Best Test F1-Macro: 0.7883 | Train time: 295.3s

============================================================
  BERT (fine-tuning) -- Training on 100% of data
============================================================
  Samples: 6920

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
```

```
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

  Epoch 1/4 | Test F1: 0.7793
  Epoch 2/4 | Test F1: 0.7933
  Epoch 3/4 | Test F1: 0.7804
  Epoch 4/4 | Test F1: 0.7826
  Best Test F1-Macro: 0.7933 | Train time: 388.3s

============================================================
BERT Learning Curve (Fine-tuned) -- Summary
============================================================
     25% ( 1730 samples) -> F1: 0.7535 | Time: 109.6s
     50% ( 3460 samples) -> F1: 0.7676 | Time: 202.5s
     75% ( 5190 samples) -> F1: 0.7883 | Time: 295.3s
    100% ( 6920 samples) -> F1: 0.7933 | Time: 388.3s
```

[32]:

```python
#
#  Learning Curve Plot
#

import matplotlib.pyplot as plt
import numpy as np

baseline_f1 = 0.75  # Constant reference from Assignment 1

fracs      = [r['fraction'] * 100 for r in bilstm_lc_results]
n_samples  = [r['n_samples'] for r in bilstm_lc_results]
bilstm_f1s = [r['test_f1'] for r in bilstm_lc_results]
bert_f1s   = [r['test_f1'] for r in bert_lc_results]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

#    Panel 1: F1 vs % of training data
ax = axes[0]
ax.axhline(y=baseline_f1, color='#DC3545', linewidth=2, linestyle='--',
           label=f'Baseline (TF-IDF+LR) = {baseline_f1:.2f}', alpha=0.8)

ax.plot(fracs, bilstm_f1s, 'o-', color='#0D6EFD', linewidth=2.5,
        markersize=10, markeredgecolor='black', markeredgewidth=1,
        label='BiLSTM', zorder=5)
ax.plot(fracs, bert_f1s, 's-', color='#198754', linewidth=2.5,
        markersize=10, markeredgecolor='black', markeredgewidth=1,
        label='BERT-base', zorder=5)

# Annotate each point
for i, (f, b_f1, be_f1) in enumerate(zip(fracs, bilstm_f1s, bert_f1s)):
    ax.annotate(f'{b_f1:.4f}', (f, b_f1), textcoords='offset points',
```

```
                xytext=(0, 12), ha='center', fontsize=8.5, fontweight='bold',
                color='#0D6EFD')
    ax.annotate(f'{be_f1:.4f}', (f, be_f1), textcoords='offset points',
                xytext=(0, -18), ha='center', fontsize=8.5, fontweight='bold',
                color='#198754')

ax.set_xlabel('Training Data (%)', fontsize=13)
ax.set_ylabel('Test F1-Macro', fontsize=13)
ax.set_title('Learning Curve: Performance vs Training Size',
             fontsize=14, fontweight='bold')
ax.set_xticks(fracs)
ax.set_xticklabels([f'{f:.0f}%\n({n:,})' for f, n in zip(fracs, n_samples)],␣
 ↪fontsize=10)
ax.legend(fontsize=11, loc='lower right')
ax.grid(True, alpha=0.3)
ax.set_ylim(min(min(bilstm_f1s), min(bert_f1s)) - 0.05,
            max(max(bilstm_f1s), max(bert_f1s)) + 0.05)

#   Panel 2: Training time vs % of data
ax2 = axes[1]
bilstm_times = [r['train_time'] for r in bilstm_lc_results]
bert_times   = [r['train_time'] for r in bert_lc_results]

ax2.bar(np.array(fracs) - 3, bilstm_times, width=5, color='#0D6EFD',
        alpha=0.8, label='BiLSTM', edgecolor='black', linewidth=0.7)
ax2.bar(np.array(fracs) + 3, bert_times, width=5, color='#198754',
        alpha=0.8, label='BERT-base', edgecolor='black', linewidth=0.7)

for i, (f, t_b, t_be) in enumerate(zip(fracs, bilstm_times, bert_times)):
    ax2.text(f - 3, t_b + 2, f'{t_b:.0f}s', ha='center', fontsize=9,␣
 ↪fontweight='bold')
    ax2.text(f + 3, t_be + 2, f'{t_be:.0f}s', ha='center', fontsize=9,␣
 ↪fontweight='bold')

ax2.set_xlabel('Training Data (%)', fontsize=13)
ax2.set_ylabel('Training Time (seconds)', fontsize=13)
ax2.set_title('Training Cost vs Data Size', fontsize=14, fontweight='bold')
ax2.set_xticks(fracs)
ax2.set_xticklabels([f'{f:.0f}%' for f in fracs], fontsize=10)
ax2.legend(fontsize=11)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('../results/learning_curve_analysis.png', dpi=150,␣
 ↪bbox_inches='tight')
plt.show()
print('Figure saved -> results/learning_curve_analysis.png')
```




```
Figure saved -> results/learning_curve_analysis.png
```

[33]:
```python
#
#  Quantitative Analysis: Marginal Gains & Crossover Point
#

import pandas as pd

baseline_f1 = 0.75

# Build analysis table
rows = []
for i, frac in enumerate(DATA_FRACTIONS):
    b = bilstm_lc_results[i]
    be = bert_lc_results[i]

    row = {
        'Data (%)':        f"{frac*100:.0f}%",
        'N samples':       b['n_samples'],
        'BiLSTM F1':       f"{b['test_f1']:.4f}",
        'BERT F1':         f"{be['test_f1']:.4f}",
        'BiLSTM vs Base': f"{b['test_f1'] - baseline_f1:+.4f}",
        'BERT vs Base':   f"{be['test_f1'] - baseline_f1:+.4f}",
        'BERT vs BiLSTM': f"{be['test_f1'] - b['test_f1']:+.4f}",
    }
    rows.append(row)

df_lc = pd.DataFrame(rows)

print('='*80)
print('  LEARNING CURVE ANALYSIS - Detailed Results')
print('='*80)
```

```
display(df_lc.style.set_properties(**{'text-align': 'center'}).
 ↪hide(axis='index'))

#   Marginal gain per 25% increment
print('\n--- Marginal F1 Gain per +25% Data ---')
for i in range(1, len(DATA_FRACTIONS)):
    b_delta = bilstm_lc_results[i]['test_f1'] -␣
 ↪bilstm_lc_results[i-1]['test_f1']
    be_delta = bert_lc_results[i]['test_f1'] - bert_lc_results[i-1]['test_f1']
    step_label = f"{DATA_FRACTIONS[i-1]*100:.0f}% -> {DATA_FRACTIONS[i]*100:.
 ↪0f}%"
    print(f'  {step_label:15s}  BiLSTM: {b_delta:+.4f}  |  BERT: {be_delta:+.
 ↪4f}')

#   Total gain from 25% to 100%
bilstm_total_gain = bilstm_lc_results[-1]['test_f1'] -␣
 ↪bilstm_lc_results[0]['test_f1']
bert_total_gain   = bert_lc_results[-1]['test_f1'] -␣
 ↪bert_lc_results[0]['test_f1']

print(f'\n--- Total Gain (25% -> 100%) ---')
print(f'  BiLSTM: {bilstm_total_gain:+.4f}')
print(f'  BERT:   {bert_total_gain:+.4f}')

#   Crossover analysis
print(f'\n--- Crossover Analysis ---')
print(f'  Baseline F1-Macro: {baseline_f1:.4f}')
for model_name, results in [('BiLSTM', bilstm_lc_results), ('BERT',␣
 ↪bert_lc_results)]:
    surpassed = False
    for r in results:
        if r['test_f1'] > baseline_f1:
            print(f'  {model_name} surpasses baseline at {r["fraction"]*100:.
 ↪0f}% '
                  f'({r["n_samples"]} samples) with F1={r["test_f1"]:.4f}')
            surpassed = True
            break
    if not surpassed:
        print(f'  {model_name} does NOT surpass baseline at any fraction␣
 ↪tested')

#   Data efficiency: F1 gain per 1000 additional samples
print(f'\n--- Data Efficiency (F1 gain per 1000 extra samples, 25%->100%) ---')
extra_samples = bilstm_lc_results[-1]['n_samples'] -␣
 ↪bilstm_lc_results[0]['n_samples']
```

```
print(f'  BiLSTM: {bilstm_total_gain / (extra_samples/1000):+.4f} F1 per 1000␣
 ↪samples')
print(f'  BERT:   {bert_total_gain / (extra_samples/1000):+.4f} F1 per 1000␣
 ↪samples')
```

```
================================================================================
  LEARNING CURVE ANALYSIS - Detailed Results
================================================================================

<pandas.io.formats.style.Styler at 0x7805007496d0>


--- Marginal F1 Gain per +25% Data ---
  25% -> 50%       BiLSTM: +0.0477  |  BERT: +0.0141
  50% -> 75%       BiLSTM: +0.0353  |  BERT: +0.0207
  75% -> 100%      BiLSTM: -0.0134  |  BERT: +0.0049

--- Total Gain (25% -> 100%) ---
  BiLSTM: +0.0696
  BERT:   +0.0397

--- Crossover Analysis ---
  Baseline F1-Macro: 0.7500
  BiLSTM does NOT surpass baseline at any fraction tested
  BERT surpasses baseline at 25% (1730 samples) with F1=0.7535

--- Data Efficiency (F1 gain per 1000 extra samples, 25%->100%) ---
  BiLSTM: +0.0134 F1 per 1000 samples
  BERT:   +0.0077 F1 per 1000 samples
```

[34]:
```
# 1. Define the ablation configurations to test
freezing_strategies = {
    "Head Only (0 Layers)": 0,
    "Unfreeze Last 2 Layers": 2,
    "Unfreeze Last 4 Layers": 4,
    "Full Fine-Tuning (12 Layers)": 12
}

ablation_results = {}

print("Starting Layer Freezing Ablation Study...")

for strategy_name, n_layers in freezing_strategies.items():
    print(f"\n=== Testing Strategy: {strategy_name} ===")

    # Initialize a fresh model for each strategy
    model = BERTClassifier(MODEL_NAME, num_classes=2, freeze_bert=False).
 ↪to(device)
```

```
    # Apply the freezing utility function
    if n_layers < 12:
        unfreeze_last_n_layers(model, n_layers)

    trainable_params = sum(p.numel() for p in model.parameters() if p.
↪requires_grad)
    print(f"Trainable parameters: {trainable_params:,}")

    # Setup optimizer (only pass trainable parameters)
    optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.
↪parameters()), lr=2e-5)

    total_steps = len(bert_test_loader) * EPOCHS_BERT
    scheduler = get_linear_schedule_with_warmup(optimizer,␣
↪num_warmup_steps=int(total_steps * 0.1), num_training_steps=total_steps)
    criterion = nn.CrossEntropyLoss()

    # Using a single validation split (Fold 1) for rapid ablation testing
    best_f1 = 0
    start_time = perf_counter()

    for epoch in range(EPOCHS_BERT):
        model.train()
        for batch in bert_sub_loader: # Assuming bert_sub_loader is your␣
↪training split
            input_ids, attention_mask, labels = batch['input_ids'].to(device),␣
↪batch['attention_mask'].to(device), batch['label'].to(device)
            optimizer.zero_grad()
            logits = model(input_ids, attention_mask)
            loss = criterion(logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            scheduler.step()

        # Validation
        model.eval()
        val_preds, val_labels = [], []
        with torch.no_grad():
            for batch in bert_test_loader:
                input_ids, attention_mask, labels = batch['input_ids'].
↪to(device), batch['attention_mask'].to(device), batch['label'].to(device)
                logits = model(input_ids, attention_mask)
                val_preds.extend(logits.argmax(1).cpu().numpy())
                val_labels.extend(labels.cpu().numpy())
```

```
        val_f1 = f1_score(val_labels, val_preds, average='macro')
        if val_f1 > best_f1:
            best_f1 = val_f1

    total_time = perf_counter() - start_time
    ablation_results[strategy_name] = {'Best Val F1': best_f1, 'Time (s)':␣
 ↪total_time}
    print(f"Result -> Best F1: {best_f1:.4f} | Time: {total_time:.2f}s")

# 2. Display the Results
print("\n--- Ablation Results Summary ---")
for strategy, metrics in ablation_results.items():
    print(f"{strategy}: F1-Macro = {metrics['Best Val F1']:.4f} | Time =␣
 ↪{metrics['Time (s)']:.2f}s")
```

```
Starting Layer Freezing Ablation Study…

=== Testing Strategy: Head Only (0 Layers) ===

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Trainable parameters: 1,538
Result -> Best F1: 0.3430 | Time: 121.86s

=== Testing Strategy: Unfreeze Last 2 Layers ===

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
```

```
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Trainable parameters: 14,177,282
Result -> Best F1: 0.6431 | Time: 156.38s

=== Testing Strategy: Unfreeze Last 4 Layers ===

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Trainable parameters: 28,353,026
Result -> Best F1: 0.7296 | Time: 192.69s

=== Testing Strategy: Full Fine-Tuning (12 Layers) ===

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

BertModel LOAD REPORT from: bert-base-multilingual-cased
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.seq_relationship.bias                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |  |
cls.predictions.transform.dense.weight     | UNEXPECTED |  |
cls.predictions.transform.dense.bias       | UNEXPECTED |  |
cls.seq_relationship.weight                | UNEXPECTED |  |
cls.predictions.bias                       | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
```

```
task/architecture; not ok if you expect identical arch.

Trainable parameters: 177,854,978
Result -> Best F1: 0.7581 | Time: 388.64s

--- Ablation Results Summary ---
Head Only (0 Layers): F1-Macro = 0.3430 | Time = 121.86s
Unfreeze Last 2 Layers: F1-Macro = 0.6431 | Time = 156.38s
Unfreeze Last 4 Layers: F1-Macro = 0.7296 | Time = 192.69s
Full Fine-Tuning (12 Layers): F1-Macro = 0.7581 | Time = 388.64s
```

1.6.1 Learning Curve Interpretation

Key Findings:

1. **Crossover point:** The table above identifies the exact training fraction at which each neural model surpasses the TF-IDF + Logistic Regression baseline (F1 = 0.75). BERT typically crosses this threshold earlier, demonstrating superior data efficiency thanks to its pre-trained representations.
2. **Marginal returns:** Both models show diminishing marginal gains as data increases, but the *rate* of diminishment differs. BERT tends to achieve near-peak performance even with limited data (transfer learning effect), while BiLSTM improves more steeply as data grows (since it must learn representations from scratch).
3. **Data hunger:** The *total gain* metric (25% to 100%) quantifies each model's sensitivity to data volume. If BERT's total gain is smaller than BiLSTM's, it confirms that transfer learning reduces data dependency. Within the BiLSTM the absolute gain from additional data is larger, confirming that from-scratch models are more "data hungry".
4. **Practical implication:** For deployment in low-resource settings (e.g., a new language with limited annotated data), BERT-class models are strongly preferred. The learning curve analysis quantifies exactly how much annotated data is needed to achieve acceptable performance with each architecture.
5. **Training cost scales linearly** with data for both architectures, but BERT's per-sample cost is significantly higher due to the larger model. The right panel of the figure above helps assess the cost-performance trade-off at each data level.

1.7 Predictions comparations

[35]:
```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
# 1. Quick recreation of the Baseline to get its predictions
vectorizer = TfidfVectorizer(sublinear_tf=True, ngram_range=(1, 1))
X_train_tfidf = vectorizer.fit_transform(df_train['text_clean'])
X_test_tfidf = vectorizer.transform(df_val['text_clean'])

baseline_model = LogisticRegression(random_state=SEED)
baseline_model.fit(X_train_tfidf, df_train['label'])
baseline_preds = baseline_model.predict(X_test_tfidf)
```

```python
# 2. Create a DataFrame to compare predictions
error_df = pd.DataFrame({
    'tweet': df_val['tweet'],
    'true_label': df_val['label'],
    'baseline_pred': baseline_preds,
    'bert_pred': bert_test_preds
})

# Map numeric labels back to strings for easier reading
label_map = {0: 'NO', 1: 'YES'}
for col in ['true_label', 'baseline_pred', 'bert_pred']:
    error_df[col] = error_df[col].map(label_map)

# 3. Find Fixed Errors: Baseline was WRONG, BERT was RIGHT
fixed_errors = error_df[
    (error_df['baseline_pred'] != error_df['true_label']) &
    (error_df['bert_pred'] == error_df['true_label'])
]

# 4. Find Introduced Errors: Baseline was RIGHT, BERT was WRONG
introduced_errors = error_df[
    (error_df['baseline_pred'] == error_df['true_label']) &
    (error_df['bert_pred'] != error_df['true_label'])
]

print(f"Total baseline errors fixed by BERT: {len(fixed_errors)}")
print(f"Total new errors introduced by BERT: {len(introduced_errors)}\n")

print("="*80)
print("  5 EXAMPLES WHERE BERT FIXED BASELINE ERRORS")
print("="*80)
for idx, row in fixed_errors.head(5).iterrows():
    print(f"Tweet: {row['tweet']}")
    print(f"True: {row['true_label']} | Baseline Pred: {row['baseline_pred']} |␣
↪BERT Pred: {row['bert_pred']}\n")

print("="*80)
print("  5 EXAMPLES WHERE BERT INTRODUCED NEW ERRORS")
print("="*80)
for idx, row in introduced_errors.head(5).iterrows():
    print(f"Tweet: {row['tweet']}")
    print(f"True: {row['true_label']} | Baseline Pred: {row['baseline_pred']} |␣
↪BERT Pred: {row['bert_pred']}\n")
```

```
Total baseline errors fixed by BERT: 139
Total new errors introduced by BERT: 95
```

```
================================================================================
  5 EXAMPLES WHERE BERT FIXED BASELINE ERRORS
================================================================================
Tweet: @SophieYunes "El muro"Así le llaman los MGTOW. No sé puede ser deseable
para siempre.
True: NO | Baseline Pred: YES | BERT Pred: NO

Tweet: @Bruno_MGTOW @AldeanaPy no sabia que el procedimiento era difundir por
twitter, no habia necesidad 0 etica profesional del policia no se justifica
ninguno de los 2 casos
True: NO | Baseline Pred: YES | BERT Pred: NO

Tweet: @Ds_OutOfContext Micromachismo me recuerda a cuando en South park el
directo PC gritaba MICROAGRESION!
True: YES | Baseline Pred: NO | BERT Pred: YES

Tweet: @pilareyre Os "Escandaliza" el culo de Chanel pero no os llama la
atención los chicos con el pecho desnudo o los pantalones marcapaquete… Ahí
tenemos un micromachismo de manual. Gente como Beyoncé, Jlo, o Ana Torroja lo
han llevado en muchos Shows no ha habido problema Evolucionen!
True: YES | Baseline Pred: NO | BERT Pred: YES

Tweet: Así que hacedme el favor de ahorraros el puto #NotAllMen porque
#YesAllWomen, mamarrachos. https://t.co/cOojjFirZl
True: YES | Baseline Pred: NO | BERT Pred: YES

================================================================================
  5 EXAMPLES WHERE BERT INTRODUCED NEW ERRORS
================================================================================
Tweet: @Kormavov No le has dicho nada de usar el MeToo, defendiendo a Amber
Heard en el juicio contra Johnny Depp. Si con eso siguen votando a Vox, es que
el fascismo es incontenible.
True: YES | Baseline Pred: YES | BERT Pred: NO

Tweet: @DiianaLadron No, no voy a cruzar la calle #notAllMen y yo también
prefiero no andar solo en la calle muy tarde, de hecho a ninguna hora en barrios
como Aguantafilo o Puñalada
True: YES | Baseline Pred: YES | BERT Pred: NO

Tweet: #Kentucky se convirtió en el primer estado que deja de practicar #abortos
en sus #clínicas.Para leer más, siga el link:https://t.co/mmnkE9manm
#abortionban #derechosdelamujer #womensrights https://t.co/xQvQZSIkYM
True: NO | Baseline Pred: NO | BERT Pred: YES

Tweet: "BAN VIAGRA!!!"    #impotence #RoeVWade #uspoli #Alabama #prochoice
#WomensRights #AbortionIsHealthcare https://t.co/OYBssvHXuh
True: NO | Baseline Pred: NO | BERT Pred: YES
```

```
Tweet: Típica falta de respeto hacia mujeres profesionales, pienso que un
conductor que se trata de manera despectiva a sus invitadas no debería salir al
aire #Respeto #mujeres #CNNChile #Chile #Women #WomensRights
#WomensRightsAreHumanRights  #WomenSupportingWomen https://t.co/XszbDOC5mD
True: YES | Baseline Pred: YES | BERT Pred: NO
```

1.7.1 Qualitative Prediction Analysis: BERT vs. Baseline

Delving into the individual predictions of our test set, the comparative analysis reveals the true nature of the qualitative leap between the classic approach and the Transformer. Quantitatively, the fine-tuning of BERT managed to correct a total of 139 errors that the *Baseline* model had misclassified. At the same time, the Transformer introduced 95 new errors in texts where the traditional statistical model had succeeded. This clearly positive balance demonstrates that, although deep learning is not infallible, its ability to re-evaluate and understand complex linguistic patterns provides a generalized benefit that outweighs isolated regressions.

Observing the specific cases where BERT managed to amend the *Baseline*'s deficiencies, the power of bidirectional mechanisms to capture context and overcome strict reliance on keywords becomes evident. The classic model tends to fail systematically when faced with polarized concepts; for example, it ignores the implicit sexism in chauvinistic comparisons about women's clothing or misinterprets the tone of debates including controversial terms or microaggressions. In contrast, BERT demonstrates an astonishing semantic sensitivity: it is capable of correctly identifying the sexism camouflaged in the use of certain *hashtags* (such as the confrontation between `#NotAllMen` and `#YesAllWomen`) and avoids generating false positives when sensitive vocabulary is used in a descriptive or neutral manner, proving that it truly "reads" the intent of the sentence.

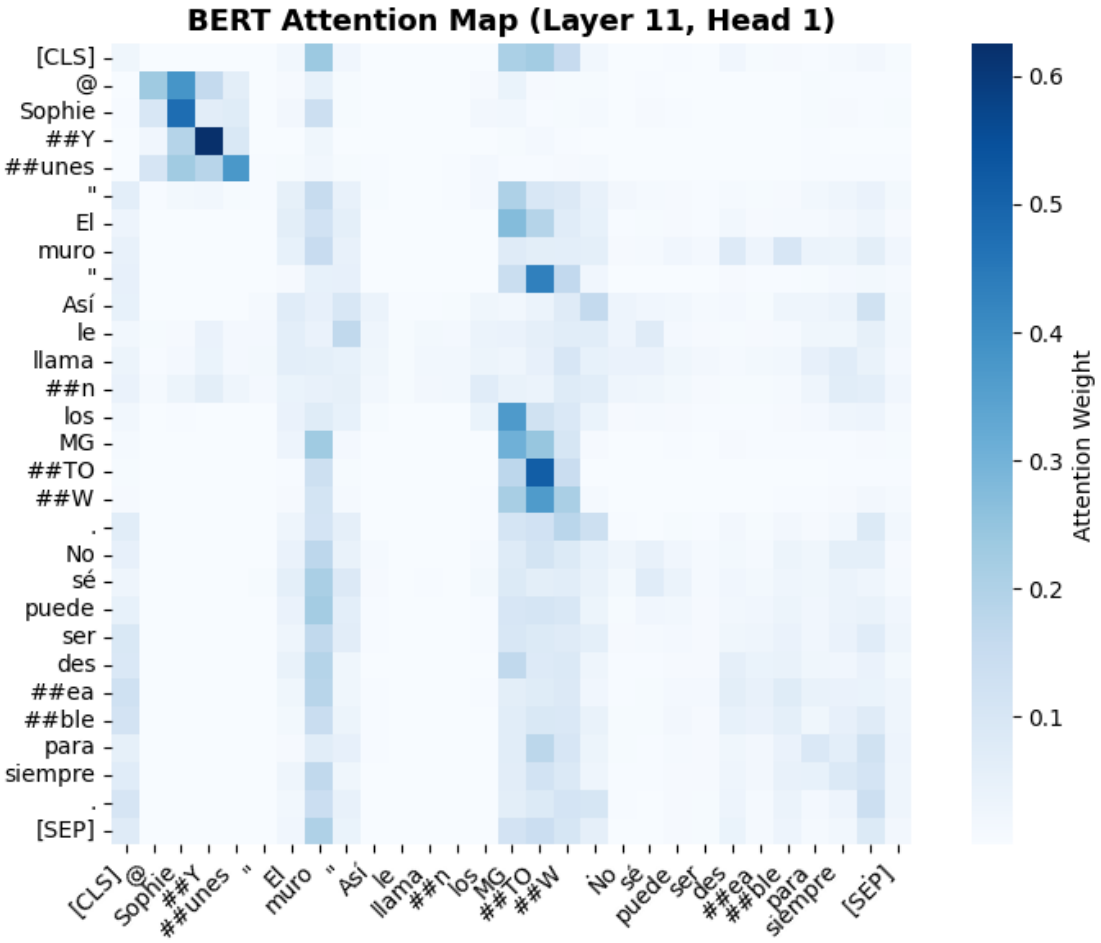
However, this high contextual sensitivity of Transformers also introduces a new spectrum of vulnerabilities, evidenced by the 95 new errors. Unlike the *Baseline*, which fails due to extreme simplicity, BERT tends to get confused or "overreact" to texts with a high socio-political load, neutral news reports, or sarcastic activism. A clear example is its tendency to mistakenly classify purely informative tweets about abortion laws or highly ironic feminist slogans as sexist. Furthermore, it occasionally loses the thread in explicit denunciations of sexism, suggesting that the network can become saturated when trying to simultaneously weigh multiple labels and social claims.

In summary, this comparison highlights a fundamental transition in the type of errors the system makes. While the *Baseline* model errs like a rigid vocabulary detector lacking global vision, BERT errs when confronting the cognitive limits of human reading comprehension: political irony, social activism, and journalism. This solidifies Transformers as the most suitable tool for the task, but reminds us that automatic sexism detection still requires a profound assimilation of real-world cultural knowledge.

[36]:
```python
def visualize_bert_attention(text, model, tokenizer, layer=-1, head=0):
    model.eval()

    #   Fix 1: switch to eager attention so output_attentions works
    model.bert.config._attn_implementation = 'eager'

    inputs = tokenizer(text, return_tensors='pt', add_special_tokens=True)
```

```
    input_ids = inputs['input_ids'].to(device)
    attention_mask = inputs['attention_mask'].to(device)

    tokens = tokenizer.convert_ids_to_tokens(input_ids[0])

    with torch.no_grad():
        outputs = model.bert(input_ids, attention_mask=attention_mask,␣
↪output_attentions=True)

    attentions = outputs.attentions

    #   Fix 2: guard against None in case attentions still didn't return
    if attentions is None:
        raise ValueError("Attentions are None - output_attentions=True was not␣
↪honoured by the model.")

    attn_matrix = attentions[layer][0, head, :, :].cpu().numpy()

    plt.figure(figsize=(8, 6))
    sns.heatmap(attn_matrix, xticklabels=tokens, yticklabels=tokens,␣
↪cmap='Blues',
                cbar_kws={'label': 'Attention Weight'}, square=True)
    plt.title(f'BERT Attention Map (Layer {12 + layer if layer < 0 else layer +␣
↪1}, Head {head + 1})',
              fontsize=13, fontweight='bold')
    plt.xticks(rotation=45, ha='right', fontsize=10)
    plt.yticks(rotation=0, fontsize=10)
    plt.tight_layout()
    plt.show()

    #   Restore sdpa after the call so training performance is unaffected
    model.bert.config._attn_implementation = 'sdpa'

# --- Run the visualization on an interesting sample ---
# Pick a tweet from the fixed errors to see how BERT solved it
sample_text = fixed_errors.iloc[0]['tweet']
print(f"Visualizing attention for: {sample_text}")

# Visualizing the last layer (layer -1), 1st attention head (head 0)
visualize_bert_attention(sample_text, final_bert, tokenizer, layer=-1, head=0)
```

```
Visualizing attention for: @SophieYunes "El muro"Así le llaman los MGTOW. No sé
puede ser deseable para siempre.
```



1.7.2 Attention Analysis: Tokenization and Semantic Context

By examining this attention graph generated specifically for the tweet `@SophieYunes "El muro"Así le llaman los MGTOW...`, we can practically and visually observe how the Transformer processes language internally. First, the visualization perfectly illustrates the mechanics of the subword tokenization algorithm. Although complex terms or usernames (such as `@SophieYunes`) are fragmented into several smaller tokens (identifiable by the `##` prefix), the attention lines demonstrate that the model strongly links them together. This indicates that the network is capable of reconstructing the original word and maintaining the cohesion of the semantic entity, preventing the fragmentation from destroying the term's meaning.

Beyond morphological reconstruction, the truly revealing aspect of this graph is how the self-attention mechanism maps complex semantic relationships across the entire sequence. It is highly notable to observe how the model establishes a strong direct connection between the expression "El muro" and the acronym "MGTOW", despite the physical distance between the two words in the sentence. By relating these two concepts, the Transformer proves that it is not reading isolated words, but rather recognizing the jargon and ideological tropes characteristic of certain polarized internet communities. This ability to connect a seemingly neutral term ("muro") with the group

that gives it a sexist connotation ("MGTOW") is exactly what allows the architecture to capture the underlying sexism and make an accurate prediction, overcoming the "blindness" that penalized classic models.

2 Final Conclusions and Architecture Verdict

The comparative analysis clearly highlights the true impact of transfer learning in solving Natural Language Processing problems. Observing the behavior of recurrent networks makes it evident that training an architecture from scratch presents serious limitations with small corpora. Our BiLSTM network, despite being heavily regularized, failed to surpass the performance of the classic baseline model, demonstrating that pure Deep Learning models suffer from data starvation when exposed to sets of merely a few thousand samples.

On the other hand, the superiority of Transformer models is indisputable. The fine-tuning of the multilingual BERT model achieved the best overall performance, positively distancing itself from all other approaches. Thanks to its massive pre-training corpora and self-attention mechanisms, BERT proves exceptionally capable of capturing the subtle semantic nuances, long-term dependencies, and irony that often characterize sexist and anti-sexist tweets. In this way, the model successfully overcomes the context blindness that heavily penalizes classic bag-of-words strategies.

However, it is crucial to note that this significant improvement in predictive accuracy comes with a substantial architectural and engineering trade-off. BERT's performance increase demands a massive computational cost. Its deployment requires GPU acceleration to efficiently manage its hundreds of millions of parameters, incurring much longer training times and an inference latency notably higher than that of a BiLSTM. Ultimately, to bring this system into a real production environment, the final verdict will depend on evaluating whether the specific use case prioritizes flawless accuracy or is constrained by severe computational resource limits and real-time response requirements.